



Rust for Game Devs

A Hands-On Handbook for Python Developers — Extended Edition

Learn modern Rust by cloning 90s classics with the Bevy engine. Ownership, the borrow checker, ECS, and design patterns — every concept taught through a real game.

Workshops: Shoot'em Up · RTS/STR · Card Game · Point & Click · City Builder · Visual Novel · FPS

Bevy 0.18

Rust 2024

June 2026

Adapted from the Rust Std Library, Rust Design Patterns, and the Rust Cookbook — re-targeted for game development.

Contents

How to Read This Handbook	7
What changed in this Extended Edition	7
Why Rust for Games?	8
Speed: where Python hurts in a game loop	8
Safety: no more 3 a.m. segfaults	8
Meet Bevy: a data-driven ECS engine	8
Setting up your toolchain	9
Rust Fundamentals	11
Variables, types & mutability	11
Functions & closures	11
Structs & enums	12
Pattern matching	12
Collections: Vec, HashMap, HashSet, BTreeMap	13
Error handling: Option & Result	14
Iterators & functional style	15
Traits & Generics	16
Traits are interfaces (and a bit more)	16
Generics: one function, many types	16
Static dispatch vs dynamic dispatch	17
Derive macros: free trait implementations	17
The std traits worth knowing early	18
Ownership, Borrowing & Lifetimes	19
The three rules	19
Borrowing: shared (&) vs mutable (&mut)	19
Lifetimes demystified	20
Smart pointers: Box, Rc, Arc	21
Interior mutability: Cell, RefCell, Mutex	21
The .clone() anti-pattern	22
Surviving the Borrow Checker	23
The errors you will actually hit	23
The borrow-checker cheat sheet	24
The "should I clone?" decision tree	25
Four patterns that end most fights	25
Bevy ECS Architecture	26
Entities, components, systems	26
Required Components: why spawn(Camera2d) just works	26
Resources: global singletons	27
Messages vs Events — the modern split	27
Queries and filters	28

Commands and recursive despawn	28
App, Plugins, States & Schedules	29
Plugins: your game's modules	29
States: menu, playing, paused	29
Ordering systems: sets and <code>.chain()</code>	30
Update vs <code>FixedUpdate</code> : simulation needs a fixed step	30
Time & timers	31
Observers & Reactive Logic	32
Two kinds of events	32
A global observer	32
Entity-targeted events	33
Per-entity observers	33
Lifecycle hooks: react to the ECS itself	34
What we covered	34
Assets, Sprites & Input	35
Loading assets	35
Sprites and the camera	35
Keyboard input	36
Mouse buttons and motion	36
Click → world position	37
A first taste of UI	37
What we covered	38
Performance & Change Detection	39
Change detection: do nothing when nothing changed	39
Data layout: structure-of-arrays for free	39
Avoiding per-frame allocations	40
Object pooling: bullets that never deallocate	40
Parallelism: <code>par_iter_mut</code>	41
What we covered	41
Design Patterns in Rust	42
Builder — readable configuration	42
Strategy — three ways, pick the Rust-iest	42
Command — undo/redo and replays	43
Newtype — make illegal states unrepresentable	44
State machine — enums as states	44
Default — sensible zero values	44
RAII guards — cleanup that can't be forgotten	45
Type-state — encode state in the type	45
What we covered	46
Workshop: Shoot'em Up	47
The plugin skeleton	47
Components and the player	47

Firing bullets	47
Enemy waves	48
Movement (shared by bullets and enemies)	48
Collision: bullet hits enemy	49
Cleanup	49
What you built	49
Workshop: Strategy — Real-Time & Turn-Based	50
Plugin and core components	50
Selection: click to world, then mark entities	50
Orders: right-click sets a destination	51
Deterministic movement in FixedUpdate	51
Economy: harvesters tick credits	52
Production: a Message channel + an Observer	52
Turn-based strategy: the other STR	53
What you built	54
Workshop: Collectible Card Game	55
Modeling cards with enums	55
Turn and phase state	55
Shuffling and drawing — with rand 0.9	56
Playing a card via the Command pattern	56
Wiring it to Bevy	57
What you built	57
Workshop: Point-and-Click Adventure	58
The verb-target interaction model	58
Resolving a click on a hotspot	58
Combining inventory items	59
Branching dialogue	60
What you built	60
Workshop: City Builder	61
The tile grid	61
Zoning: paint tiles, recolor only what changed	61
Growth: a deterministic FixedUpdate tick	62
Budget	63
A note on data layout at city scale	63
What you built	63
Workshop: Visual Novel	64
The script as data	64
Advancing and branching	64
Conditional branching on variables	65
The typewriter effect	65
Wiring portraits and the box	66
What you built	66

Workshop: First-Person Shooter	67
Scene setup: camera, light, floor	67
Mouse look	67
Movement relative to facing	68
Hitscan shooting: ray vs sphere	69
What you built	70
The Standard Library for Games	71
Collections beyond Vec	71
Strings and parsing	72
File I/O: saving and loading	72
Error handling that scales	73
Concurrency: threads for the heavy stuff	73
Math and randomness	74
What we covered	74
Practical Recipes	75
Recipe: versioned save files with migration	75
Recipe: human-editable config with RON or TOML	75
Recipe: logging instead of println!	76
Recipe: timing a system (lightweight profiling)	76
Recipe: deterministic RNG for replays	77
Recipe: loading a data table	77
What we covered	78
Testing & Debugging Bevy	79
Testing pure functions	79
Testing systems by stepping an App	80
Testing messages and state transitions	80
The Bevy traps that bite at runtime	81
A quick debugging toolbox	82
What we covered	82
Appendix A · Python → Rust Cheat Sheet	83
Syntax and types	83
Control flow and functions	83
Collections	83
Errors and absence	84
Classes vs structs	84
Appendix B · Bevy 0.18 API Quick Reference	85
App and scheduling	85
Entities and components	85
Queries	85
Messages vs Events (the big one)	86
Renames cheat sheet (old → new)	86

Appendix C · Cargo.toml Recipes	87
2D action (shmup, point-and-click, card game, visual novel)	87
2D simulation (city builder, RTS)	87
3D (FPS and beyond)	87
The fast-iteration profile (every project)	87
Appendix D · The Challenge Ladder	89
Day 1 — spawn 100 entities at random positions	89
Day 2 — move them by a Velocity each frame	89
Day 3 — despawn entities that leave the screen	90
Days 4-10 — targets and hints	90
Appendix E · Where to Go Next	91
Official and canonical	91
Community references	91
Crates worth knowing	91
A closing word	91

How to Read This Handbook

This is a guided tour of **modern Rust** for someone who already thinks in Python and wants to build games. Every concept arrives wrapped in a real game development problem, and every code sample targets **Bevy 0.18** (released January 2026) on the **Rust 2024** edition.

You do not need to read front-to-back. Three reading paths:

- **The crash course.** Chapters 1–6 give you enough Rust and enough Bevy to spawn things on screen and move them around. Stop there and start building.
- **The wall.** Chapters 4–5 are *the* chapters. Ownership and the borrow checker are the single biggest reason Python developers bounce off Rust. We spend real time there.
- **The cookbook.** Chapters 11–21 are reference material: design patterns, one workshop per game genre, the standard library, practical recipes, and testing. Dip in as needed.

THE TRANSLATOR'S MINDSET

You are not a beginner programmer. You are a *fluent Python speaker learning a second language*. The fastest way through is to keep asking "how would I say this in Python?" and then learn where the two languages disagree. The **For the Python developer** boxes do exactly that.

What changed in this Extended Edition

Earlier drafts of this handbook targeted an older mental model of Bevy. The engine moved, and a few examples would no longer compile on 0.18. Here is what this edition corrects and adds, so you are not learning yesterday's API:

- **Events were split into *Messages* and *Events*.** Since Bevy 0.17, the old buffered `EventWriter` / `EventReader` pattern is called a **Message** (`#[derive(Message)]`, `MessageWriter`, `MessageReader`, `app.add_message`). The name **Event** is now reserved for *observer* events that you `trigger` and react to immediately. Chapter 6 teaches the new split and Chapter 8 is a brand-new chapter on observers.
- **`Query::single` returns a `Result`.** It no longer panics silently; you handle the `Ok` / `Err` or use `?`. Systems can now return `Result`, which makes `?` ergonomic.
- **`despawn()` is recursive.** Despawning an entity despawns its children too; `despawn_recursive` is gone. This follows the ECS relationships rework.
- **Required Components.** `commands.spawn(Camera2d)` works because a component can *require* others. We explain why, since it changes how you spawn things.
- **New chapters and sections:** Traits & Generics (Ch. 3), an expanded Borrow-Checker Survival chapter (Ch. 5), Observers (Ch. 8), Change Detection & Performance (Ch. 10), two new design patterns — RAII / `Drop` guards and the type-state pattern (Ch. 11) — Testing & Debugging Bevy (Ch. 21), and an Advent-of-Code-style challenge ladder (Appendix D).
- **Corrected example bugs**, e.g. a leaderboard printer that reported the same rank for every entry now uses `.enumerate()`.

A WORD ON A MOVING TARGET

Bevy is pre-1.0 and makes breaking changes every ~3 months. Code here compiles on 0.18. When you upgrade, keep the official migration guide open — it is searchable and lists every rename. Appendix B is a 0.18 quick reference you can diff against.

PART I · WHY RUST, WHY BEVY

Why Rust for Games?

Rust gives you C++-level performance with memory safety guaranteed *at compile time*. No segfaults, no data races, no garbage-collector pauses in your frame loop. The price is a compiler that refuses to build code it cannot prove is safe — which feels hostile at first and becomes your most valuable teammate once you understand what it is protecting you from.

Speed: where Python hurts in a game loop

A game loop runs ≥ 60 times per second. Inside each frame you update physics, AI, input, audio, and rendering. In CPython, three things quietly tax every iteration: attribute lookups go through dictionaries, the GIL serializes threads, and the garbage collector can pause you mid-frame. A Pygame loop over 10,000 entities with collision often dips under 30 fps. The same logic in Bevy runs at hundreds of fps because Rust compiles to native code, Bevy stores components in cache-friendly arrays, and there is no GC.

Python — 10,000 sprites in Pygame

```
for entity in entities:           # 10,000 iterations
    entity.x += entity.vx * dt     # dict lookup per attribute
    entity.y += entity.vy * dt     # GC may pause mid-frame
    if check_collision(entity):    # dynamic dispatch
        handle_hit(entity)
```

Rust — the same work as a Bevy system

```
fn move_entities(time: Res<Time>, mut query: Query<(&mut Transform, &Velocity)>) {
    let dt = time.delta_secs();
    for (mut tf, vel) in &mut query {
        tf.translation.x += vel.0.x * dt; // contiguous memory, zero allocation
        tf.translation.y += vel.0.y * dt;
    }
}
```

Safety: no more 3 a.m. segfaults

In Python you never think about memory; the cost is unpredictable pauses. In C++ you get speed but risk dangling pointers and use-after-free. Rust's ownership system is the third option: the compiler proves your memory access is valid before the program ever runs. If it compiles, you do not have null dereferences, use-after-free, or data races. That guarantee is the entire reason the borrow checker exists, and it is why Chapters 4–5 matter so much.

Meet Bevy: a data-driven ECS engine

Bevy is a free, open-source engine written in Rust. It uses an **Entity Component System** (ECS), which inverts the object-oriented model you may know from Unity or Godot. Instead of inheritance trees, you compose entities out of small data **components** and write **systems** (plain functions) that operate on whatever components they ask for. We unpack ECS in Chapter 6; for now, here is the smallest complete Bevy 0.18 program.


```
use bevy::prelude::*;

fn main() {
    App::new()
        .add_plugins(DefaultPlugins)
        .add_systems(Startup, setup)           // runs once
        .add_systems(Update, greet)           // runs every frame
        .run();
}

#[derive(Component)]
struct Player { name: String, health: f32 }

fn setup(mut commands: Commands) {
    commands.spawn(Camera2d);                // a 2D camera
    commands.spawn(Player { name: "Ferris".into(), health: 100.0 });
}

fn greet(query: Query<&Player>) {
    for player in &query {
        info!("Player {} has {} HP", player.name, player.health);
    }
}
```

READING THAT FILE

`#[derive(Component)]` is like a class decorator that registers the type with the engine. `fn setup(...)` is just a function — Bevy calls it for you and *injects* its arguments based on their types. That dependency-injection-by-type idea is the heart of Bevy, and it is the opposite of Python where you import and call explicitly.

Setting up your toolchain

```
# Install Rust (all platforms)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Create a project and add Bevy
cargo new my_game
cd my_game
cargo add bevy@0.18
cargo run                # first build downloads + compiles deps (a few minutes)
```

The first Bevy compile is slow because it builds the whole engine once. After that, incremental builds are fast — *if* you optimize your dependencies in dev mode. Bevy 0.18 also ships **feature collections** so you only compile the parts you use.

Cargo.toml – fast iterative dev, 2D-only features

```
[package]
name = "my_game"
version = "0.1.0"
edition = "2024"

[dependencies]
# Pull only what a 2D game needs instead of the whole default feature set.
bevy = { version = "0.18", default-features = false, features = [
```

```

    "2d",          # high-level 2D collection (sprites, 2d camera, ...)
    "bevy_window",
    "bevy_winit",
    "png",
    "x11",         # or "wayland" on Linux; omit on macOS/Windows
] }
rand = "0.9"
serde = { version = "1", features = ["derive"] }
serde_json = "1"

# Your code stays debuggable, but dependencies are optimized → big speedup.
[profile.dev]
opt-level = 1

[profile.dev.package."*"]
opt-level = 3

```

ITERATE FASTER

For day-to-day work, also try the dynamic-linking feature (`features = ["dynamic_linking"]`) and the `cranelift` codegen backend. They cut rebuild times substantially. Turn them off for release builds. If you only ever ship 2D, leaving `default-features = false` and opting into the `2d` collection avoids compiling the 3D renderer entirely.

EDITION VS VERSION

`edition = "2024"` is the Rust *language* edition and is unrelated to your crate version or Bevy's version. Editions are opt-in language epochs; 2024 is the current one and what every example here assumes.

PART II · RUST FUNDAMENTALS THROUGH A GAME LENS

Rust Fundamentals

This chapter maps core Rust to game scenarios. If you know Python, the left column will feel familiar; the right column shows the Rust equivalent. The mental shifts that matter most: everything is immutable by default, there are no exceptions, and the type system is your friend rather than red tape.

Variables, types & mutability

In Python every binding is mutable. In Rust every binding is **immutable by default**; you opt into mutation with `mut`. This sounds annoying and turns out to prevent whole categories of state-management bugs, because the compiler now knows exactly what can change.

Python

```
player_hp = 100          # mutable
MAX_HP = 100             # "constant" by convention only
MAX_HP = 200             # ...nothing stops you
```

Rust

```
let player_hp: f32 = 100.0;    // immutable
// player_hp = 90.0;          // ERROR: cannot assign twice

let mut hp: f32 = 100.0;       // opt into mutability
hp -= 10.0;                    // OK

const MAX_HP: f32 = 100.0;     // truly constant, enforced
let name: &str = "Hero";       // borrowed string slice
let owned: String = String::from("Hero"); // owned, heap-allocated
```

&STR VS STRING

Think of `String` as a Python `str` you own and can grow, and `&str` as a *view* into someone else's text (a read-only slice). Function arguments usually take `&str` (cheap, borrows); struct fields that must own their text use `String`. This distinction is the gateway drug to understanding ownership.

Functions & closures

Functions need type annotations on parameters and return types. The last expression in a block is its value — no `return` needed. Closures (lambdas) are everywhere in Bevy.

Python

```
def apply_damage(hp: float, dmg: float) -> float:
    return max(0.0, hp - dmg)

enemies.sort(key=lambda e: e.distance)
```

Rust

```
fn apply_damage(hp: f32, dmg: f32) -> f32 {
    (hp - dmg).max(0.0)           // no semicolon → this is the return value
}
```

```
enemies.sort_by(|a, b| a.distance.total_cmp(&b.distance));
```

SORTING FLOATS

`f32` is not totally ordered (because `NaN`), so there is no `Ord`. Use `total_cmp` (an actual total order) or `partial_cmp(...).unwrap()` if you are certain there are no `NaN`s. The draft handbooks used `partial_cmp().unwrap()`; `total_cmp` is the safer modern choice.

Structs & enums

Python uses classes for everything. Rust splits the idea: **structs** hold data, **enums** are tagged unions (a value that is *one of* several shapes), and `impl` blocks add methods. There is no inheritance — composition via traits replaces it (Chapter 3).

Rust – a shoot'em-up power-up (R-Type / Gadius)

```
#[derive(Component)]
struct Spaceship { speed: f32, fire_rate: f32 }

#[derive(Component)]
struct Health(f32); // tuple struct – the newtype pattern (Ch. 11)

#[derive(Component, Clone, Copy, PartialEq)]
enum PowerUp {
    SpeedBoost,
    TripleShot,
    Shield { duration: f32 }, // a variant can carry data
    Missile { damage: f32, homing: bool },
}

impl PowerUp {
    fn label(&self) -> &str {
        match self {
            PowerUp::SpeedBoost      => "Speed boost",
            PowerUp::TripleShot       => "Triple shot",
            PowerUp::Shield { .. }    => "Shield",
            PowerUp::Missile { .. }   => "Homing missile",
        }
    }
}
```

Pattern matching

`match` is like Python's `match` (3.10+) but **exhaustive**: the compiler forces you to handle every variant. For game state machines, where forgetting a state is a classic bug, this is a superpower.

Rust – adventure-game state transitions (Monkey Island)

```
#[derive(States, Debug, Clone, PartialEq, Eq, Hash, Default)]
enum GameState {
    #[default] MainMenu,
    Exploring,
    Inventory,
    Dialogue,
    Puzzle { id: u32 },
}
```

```

}

fn transition(
    state: Res<State<GameState>>,
    mut next: ResMut<NextState<GameState>>,
    keys: Res<ButtonInput<KeyCode>>,
) {
    match state.get() {
        GameState::MainMenu if keys.just_pressed(KeyCode::Enter) =>
            next.set(GameState::Exploring),
        GameState::Exploring if keys.just_pressed(KeyCode::KeyI) =>
            next.set(GameState::Inventory),
        GameState::Inventory if keys.just_pressed(KeyCode::Escape) =>
            next.set(GameState::Exploring),
        GameState::Puzzle { id } => info!("solving puzzle #{id}"),
        _ => {} // the compiler made us acknowledge the rest
    }
}

```

Collections: Vec, HashMap, HashSet, BTreeMap

`Vec<T>` is Python's `list`; `HashMap<K, V>` is `dict`; `HashSet<T>` is `set`; `BTreeMap<K, V>` is a *sorted* map (great for leaderboards and timelines). The big difference is that Rust collections own their data and enforce borrowing rules.

Rust – a card deck (Magic-style)

```

use std::collections::HashMap;

#[derive(Clone)]
struct Card { name: String, mana_cost: u32, attack: i32 }

#[derive(Resource)]
struct Deck { cards: Vec<Card> }

#[derive(Resource)]
struct CardDatabase { templates: HashMap<String, Card> } // name -> template

impl Deck {
    fn draw(&mut self) -> Option<Card> {
        self.cards.pop() // Some(card) or None – no IndexError
    }
    fn castable(&self, mana: u32) -> Vec<&Card> {
        self.cards.iter().filter(|c| c.mana_cost <= mana).collect()
    }
}

```

The `entry` API: Python's `dict.setdefault`, but better

A constant beginner question is "how do I increment a count in a map, inserting 0 if the key is missing?" In Python you reach for `defaultdict` or `setdefault`. Rust's `entry` API is the idiomatic, allocation-light answer and it shows up constantly in game code (tallying loot, counting tiles by type, grouping units).

Python

```
loot = {}
for drop in drops:
    loot[drop] = loot.get(drop, 0) + 1
```

Rust

```
use std::collections::HashMap;

let mut loot: HashMap<&str, u32> = HashMap::new();
for drop in &drops {
    *loot.entry(drop).or_insert(0) += 1;    // insert 0 if absent, then add
}

// Grouping: push into a Vec stored per key.
let mut by_type: HashMap<ZoneType, Vec<Entity>> = HashMap::new();
by_type.entry(zone).or_default().push(entity);
```

Error handling: Option & Result

Rust has **no exceptions**. A function that might find nothing returns `Option<T>` (`Some` / `None`); a function that might fail returns `Result<T, E>` (`Ok` / `Err`). The `?` operator propagates the error/`None` upward, replacing try/except plumbing.

Python – loading a save

```
try:
    with open("save.json") as f:
        data = json.load(f)
except FileNotFoundError:
    data = default_save()
except json.JSONDecodeError:
    data = default_save()
```

Rust – loading a save

```
#[derive(Debug)]
enum SaveError { NotFound, Corrupt }

fn load_save(path: &str) -> Result<SaveData, SaveError> {
    let text = std::fs::read_to_string(path).map_err(|_| SaveError::NotFound)?;
    let data = serde_json::from_str(&text).map_err(|_| SaveError::Corrupt)?;
    Ok(data)    // ? returns early on the first error
}

match load_save("save.json") {
    Ok(data) => apply_save(data),
    Err(SaveError::NotFound) => use_default_save(),
    Err(SaveError::Corrupt) => { warn!("corrupted save"); use_default_save() }
}
```

UNWRAP() IS A LANDMINE

`option.unwrap()` and `result.unwrap()` extract the value but **panic** (crash) on `None` / `Err`. Fine in prototypes and tests; in shipped code prefer `?`, `match`, or `unwrap_or(default)`. Never `unwrap()` data that came from a file or the network.

Iterators & functional style

Python has comprehensions and generators; Rust has iterators with chainable adapters (`map`, `filter`, `take`, `fold`). They are **zero-cost**: the compiler fuses them into a tight loop equivalent to hand-written C.

Rust – five nearest healthy threats (Command & Conquer)

```
fn nearest_threats(me: Vec2, enemies: &[(Entity, Vec2, Health)]) -> Vec<Entity> {
    let mut scored: Vec<(Entity, f32)> = enemies.iter()
        .filter(|(_, _, hp)| hp.0 > 0.0)
        .map(|(e, pos, _)| (*e, pos.distance(me)))
        .collect();
    scored.sort_by(|a, b| a.1.total_cmp(&b.1));
    scored.into_iter().take(5).map(|(e, _)| e).collect()
}
```

COMPREHENSION ↔ ITERATOR CHAIN

`[f(x) for x in xs if cond(x)]` becomes `xs.iter().filter(|x| cond(x)).map(f).collect()`.

The `.collect()` at the end is what materializes the result into a `Vec` — until then, nothing has actually run (iterators are lazy, like generators).

Traits & Generics

Rust has no classes and no inheritance, so the question "how do I share behavior between types?" has a different answer than in Python: **traits**. A trait is an interface — a set of methods a type promises to provide. Combined with **generics**, traits let you write code that works over many types while staying fully type-checked and zero-cost.

Traits are interfaces (and a bit more)

Python — duck typing / ABC

```
class Damageable:
    def take_damage(self, amount): ...

class Enemy(Damageable):
    def take_damage(self, amount):
        self.hp -= amount
```

Rust — a trait with a default method

```
trait Damageable {
    fn hp_mut(&mut self) -> &mut f32;

    // Default method: implementors get this for free unless they override it.
    fn take_damage(&mut self, amount: f32) {
        let hp = self.hp_mut();
        *hp = (*hp - amount).max(0.0);
    }
    fn is_dead(&self) -> bool;
}

struct Enemy { hp: f32 }

impl Damageable for Enemy {
    fn hp_mut(&mut self) -> &mut f32 { &mut self.hp }
    fn is_dead(&self) -> bool { self.hp <= 0.0 }
    // take_damage is inherited from the default.
}
```

TRAITS VS BASE CLASSES

A trait is closer to a Python *Protocol* or ABC than to a base class: it describes capability, not identity. A type can implement many traits, and you can implement *your* trait for types you do not own (within the orphan rule). There is no `super().__init__` and no shared fields — only shared behavior.

Generics: one function, many types

Generics are like Python's duck typing made explicit and checked. A generic function says "I work for any type `T` that can do these things," and the *these things* are trait bounds.

```
// Works for any T that can be compared and copied — entities, scores, tile ids...
fn highest<T: PartialOrd + Copy>(items: &[T]) -> Option<T> {
    items.iter().copied().reduce(|a, b| if a > b { a } else { b })
}
```



```
let top = highest(&[40_u32, 99, 12, 73]); // Some(99)
```

The `where` clause keeps long bound lists readable:

```
fn render_all<T>(items: &[T])
where
    T: std::fmt::Display + Clone,
{
    for item in items { println!("{item}"); }
}
```

Static dispatch vs dynamic dispatch

Two ways to be generic over behavior, with a real performance tradeoff:

- **Generics / `impl Trait`** — *static dispatch*. The compiler stamps out a specialized copy per concrete type (monomorphization). Zero runtime cost, larger binary. Use this by default.
- **Trait objects (`Box<dyn Trait>`, `&dyn Trait`)** — *dynamic dispatch*. One copy, the method call goes through a vtable pointer at runtime (like a Python method call). Use this when you need to store *different* concrete types in the same collection.

Rust — AI behaviors, both ways (Command & Conquer)

```
trait AiBehavior {
    fn decide(&self, world: &WorldState) -> AiAction;
}

struct Aggressive;
struct Defensive;
impl AiBehavior for Aggressive { fn decide(&self, w: &WorldState) -> AiAction { /* ... */ AiAction::Attack } }
impl AiBehavior for Defensive { fn decide(&self, w: &WorldState) -> AiAction { /* ... */ AiAction::Defend } }

// Static: the type is known; fastest. Good when one commander has one behavior type.
fn run_static(behavior: &impl AiBehavior, w: &WorldState) -> AiAction { behavior.decide(w) }

// Dynamic: store a heterogeneous list of behaviors behind one type.
let squad: Vec<Box<dyn AiBehavior>> = vec![Box::new(Aggressive), Box::new(Defensive)];
for ai in &squad { let _ = ai.decide(&world); }
```

WHICH TO REACH FOR IN BEVY

In an ECS you often do not need either — you encode the behavior as a *component* and write a system per behavior, letting the scheduler pick which entities to process. Reach for `dyn` only when runtime polymorphism is genuinely the cleanest design (Chapter 11's Command pattern is a good case).

Derive macros: free trait implementations

`#[derive(...)]` auto-generates trait impls so you do not hand-write boilerplate. The ones you will use constantly:

Derive	What it gives you	Python analogue
<code>Debug</code>	<code>{:?}</code> formatting for logs	<code>__repr__</code>
<code>Clone</code>	explicit <code>.clone()</code> deep copy	<code>copy.deepcopy</code>
<code>Copy</code>	implicit bitwise copy (small types)	value semantics for ints
<code>Default</code>	a <code>T::default()</code> constructor	default <code>__init__</code> args
<code>PartialEq</code> , <code>Eq</code>	<code>==</code> comparison	<code>__eq__</code>
<code>Hash</code>	usable as a <code>HashMap</code> / <code>HashSet</code> key	<code>__hash__</code>
<code>PartialOrd</code> , <code>Ord</code>	<code><</code> , sorting	<code>__lt__</code> / <code>functools.total_ordering</code>

```
#[derive(Component, Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct TileCoord { x: i32, y: i32 } // now usable as a HashMap key, comparable, printable
```

COPY IS ONLY FOR SMALL, CHEAP TYPES

Derive `Copy` only for types that are trivially bit-copyable (numbers, small structs of numbers). A `String` or `Vec` field forbids `Copy` because copying them would duplicate a heap allocation silently — exactly the kind of hidden cost Rust wants you to make explicit with `.clone()`.

The std traits worth knowing early

- **From** / **Into** — conversions. Implement `From<A> for B` and you get `B::from(a)` and `a.into()` for free. This is why you see `"Hero".into()` to make a `String` from a `&str`.
- **Default** — a canonical "empty/zero" value; pairs with struct update syntax (`..Default::default()`) to override only the fields you care about.
- **Iterator** — implement `next()` and you unlock the entire adapter ecosystem (`map`, `filter`, `take`, ...) on your own type, such as a spawn-wave generator.
- **Display** — human-facing `{}` formatting (Debug is for developers, Display is for users).

```
struct Mana(u32);
impl From<u32> for Mana { fn from(n: u32) -> Self { Mana(n) } }

let cost: Mana = 3.into(); // From → Into for free
```

PART III · THE BIG WALL — OWNERSHIP

Ownership, Borrowing & Lifetimes

This is *the* chapter. Ownership is what makes Rust unique and is the number-one reason Python developers get frustrated, because in Python you never think about who owns what — the garbage collector does. Read this slowly. Every example is a real game scenario.

The three rules

OWNERSHIP IN THREE SENTENCES

1. Each value has exactly one owner. **2.** When the owner goes out of scope, the value is dropped (its memory freed). **3.** Ownership can be *moved* to a new owner, or temporarily *borrowed*, but the rules above never break.

The very first thing that trips up Python developers: assignment **moves** ownership instead of creating an alias.

Python — both names point at one list

```
crew = ["Guybrush", "LeChuck"]
pirates = crew          # alias: both work
print(crew, pirates)    # fine
```

Rust — assignment moves ownership

```
let crew = vec!["Guybrush", "LeChuck", "Elaine"];
let pirates = crew;           // ownership MOVES into `pirates`
// println!("{:?}", crew);    // ERROR[E0382]: borrow of moved value: `crew`
println!("{:?}", pirates);    // OK — pirates owns it now
```

The compiler error reads `error[E0382]: borrow of moved value`. The fix is to **borrow** (`&crew`) or, as a last resort, **clone** (`crew.clone()`).

WHY MOVE AT ALL?

Moving is how Rust guarantees there is never a second owner who might free the same memory or mutate it behind your back. It is the compile-time replacement for the garbage collector. Small `Copy` types (numbers, `bool`, small structs) are *copied* instead of moved, so this only bites with heap-owning types like `String`, `Vec`, and most structs.

Borrowing: shared (`&`) vs mutable (`&mut`)

Instead of giving away ownership, you can lend a reference. Borrowing comes in two flavors, and the rule governing them is Rust's secret weapon against data races:

THE GOLDEN RULE

At any moment you may have **either** any number of shared references (`&T`, read-only) **or** exactly one mutable reference (`&mut T`, read-write) — **never both at once**. This single rule is what makes data races impossible and iterator-invalidation bugs disappear.

Rust — borrowing a city (SimCity 2000)

```
#[derive(Resource)]
struct City { population: u32, treasury: f64 }

fn show_stats(city: &City) { // shared borrow: read-only
    info!("pop {} | ${:.0}", city.population, city.treasury);
    // city.treasury += 1.0; // ERROR: cannot mutate through &
}

fn collect_taxes(city: &mut City) { // mutable borrow: exclusive
    city.treasury += city.population as f64 * 10.0;
}

fn main_demo() {
    let mut city = City { population: 1000, treasury: 50_000.0 };
    show_stats(&city); // shared
    show_stats(&city); // many shared borrows are fine
    collect_taxes(&mut city); // exclusive borrow – and the shared ones are done
    show_stats(&city);

    // let r = &city;
    // let w = &mut city; // ERROR[E0502]: cannot borrow as mutable
    // info!("{}", r.population); // while also borrowed as immutable
}
```

Lifetimes demystified

A lifetime annotation tells the compiler *how long a reference stays valid*. Most of the time Rust infers them (lifetime elision) and you write none. You only annotate when a function returns a reference and the compiler cannot tell which input it borrows from.

Rust – dialogue lines that borrow from a script (Tokimeki Memorial)

```
// `a` says: a DialogueLine may not outlive the script text it points into.
struct DialogueLine<'a> {
    speaker: &'a str,
    text: &'a str,
    emotion: Emotion, // owned data needs no lifetime
}

fn line_at<'a>(script: &'a [String], i: usize) -> Option<DialogueLine<'a>> {
    script.get(i).map(|s| DialogueLine {
        speaker: "Shiori",
        text: s.as_str(),
        emotion: Emotion::Happy,
    })
}
```

THE ONE QUESTION TO ASK

When a lifetime error appears, ask: *does the reference I am returning outlive the data it points to?* If yes, the borrow is unsound and the compiler is right. The fix is almost always to **return an owned value** (`String`) instead of (`&str`) or to **keep the source data alive longer** (store it one scope up). You rarely need to fight with `'a` directly.

Smart pointers: Box, Rc, Arc

Sometimes you genuinely need heap allocation or shared ownership (Python's default). Rust gives you explicit tools so the cost is visible:

Pointer	Ownership	Threads	Use for
<code>Box<T></code>	single, heap	—	large values; recursive types; trait objects
<code>Rc<T></code>	shared, counted	single-threaded	many readers of one immutable value
<code>Arc<T></code>	shared, counted	thread-safe	the same, across threads

Rust – a texture atlas shared by many sprites

```
use std::rc::Rc;

let atlas = Rc::new(load_atlas("sprites.png")); // one allocation
let a = Rc::clone(&atlas);                    // cheap: just bumps a counter
let b = Rc::clone(&atlas);                    // same data, no copy
// freed automatically when the last Rc is dropped
```

IN BEVY YOU RARELY NEED RC/ARC

The ECS already owns and manages entity and component lifetimes for you, and Bevy assets (images, sounds, meshes) come behind `Handle<T>`, which is reference-counted internally. If you reach for `Rc` / `Arc` inside ECS code, stop and ask whether a component, a resource, or an asset handle is the more idiomatic home for that data.

Interior mutability: `Cell`, `RefCell`, `Mutex`

The golden rule is checked at compile time. Sometimes you need to mutate through a shared reference and prove safety at *runtime* instead — that is **interior mutability**.

- `Cell<T>` / `RefCell<T>` — single-threaded. `RefCell` enforces the borrow rules at runtime and **panics** if you break them. Often paired as `Rc<RefCell<T>>`.
- `Mutex<T>` / `RwLock<T>` — thread-safe, usually as `Arc<Mutex<T>>`.

```
use std::{rc::Rc, cell::RefCell};

let shared_score = Rc::new(RefCell::new(0_u32));
let viewer = Rc::clone(&shared_score);

*shared_score.borrow_mut() += 100; // mutate through a shared handle
println!("score = {}", viewer.borrow()); // read elsewhere
```

REFCELL MOVES BUGS FROM COMPILE TIME TO CRASH TIME

A double `borrow_mut()` that the borrow checker would have rejected becomes a *runtime panic* with `RefCell`. You have traded a compile error for a crash. Use interior mutability only when the design truly needs it, and in Bevy prefer ECS-native solutions (components, resources, `Commands`) which give you back the compile-time guarantees.

| The `.clone()` anti-pattern

When the borrow checker fights you, the tempting escape is to sprinkle `.clone()` everywhere. It compiles — and silently copies data, every frame, forever.

DEATH BY A THOUSAND CLONES

Cloning a `Vec` of 10,000 enemies each frame to satisfy the borrow checker is 10,000 heap allocations *per frame*. It is the Rust equivalent of writing `copy.deepcopy(obj)` on every line. The next chapter is entirely about how to *restructure* instead of clone.

Surviving the Borrow Checker

The compiler feels harsh because it refuses to compile code it cannot *prove* safe. But every error it throws is a bug it just caught — a bug that would have been a segfault or silent memory corruption in C++. This chapter is your field manual: the errors you will actually hit, the fix for each, and the handful of patterns that make borrow-checker fights vanish.

The errors you will actually hit

E0382 — use of moved value

You used a value after its ownership moved.

```
let enemies = vec![Enemy::default(); 100];
let wave = enemies;           // moved here
// println!("{}", enemies.len()); // ERROR[E0382]
```

Fix: borrow instead of move (`&enemies`), or restructure so you do not need both names. Clone only if you genuinely need two independent copies.

E0499 — cannot borrow as mutable more than once

Two `&mut` to the same data at the same time — classic when iterating and modifying.

```
fn update_units(units: &mut Vec<Unit>) {
    for unit in units.iter_mut() {           // first &mut borrow
        if unit.hp <= 0.0 {
            units.retain(|u| u.hp > 0.0); // ERROR[E0499]: second &mut borrow
        }
    }
}
```

Fix — separate the read phase from the write phase (the single most useful technique in all of Rust game code):

```
fn update_units(units: &mut Vec<Unit>) {
    // Phase 1 (read): figure out *what* to change.
    let dead: Vec<usize> = units.iter().enumerate()
        .filter(|(_, u)| u.hp <= 0.0)
        .map(|(i, _)| i)
        .collect();
    // Phase 2 (write): apply it. Remove back-to-front so indices stay valid.
    for i in dead.into_iter().rev() {
        units.swap_remove(i);
    }
}
```

Even simpler when you just want to drop the dead: `units.retain(|u| u.hp > 0.0);` does the whole thing in one safe call.

E0502 — immutable borrow while mutably borrowed

```
let mut units = vec![Unit::default()];
let first = &units[0];           // shared borrow
units.push(Unit::default());    // ERROR[E0502]: needs &mut while `first` is alive
println!("{first:?}");
```

Fix: finish using the shared reference before mutating, or store an **index** (`0`) instead of a reference and re-index after the mutation.

E0597 — borrowed value does not live long enough

```
let name_ref;
{
    let name = String::from("Guybrush");
    name_ref = &name;           // ERROR[E0597]: `name` dropped at the brace
}
println!("{name_ref}");
```

Fix: move the owned value to the outer scope, or return an owned `String`.

E0106 / returning a reference to a local

```
fn player_name() -> &str {      // ERROR[E0106]: missing lifetime / dangling
    let name = String::from("Guybrush");
    &name                       // `name` dies at function end
}
```

Fix: return the owned value — `fn player_name() -> String { String::from("Guybrush") }`.

E0277 — trait bound not satisfied

Usually "this type can't be hashed/compared/printed." Example: using a `struct` as a `HashMap` key without `#[derive(PartialEq, Eq, Hash)]`. **Fix:** add the derives (Chapter 3).

The borrow-checker cheat sheet

Situation	Reach for
Read the same data in two places	<code>&</code> shared borrow
Modify data in place	<code>&mut</code> exclusive borrow
Iterate and mutate a collection	collect indices, then mutate (or <code>retain</code> / <code>drain</code>)
Two <code>&mut</code> into different parts of one slice	<code>split_at_mut</code>
Swap a field out to work on it	<code>std::mem::take</code> / <code>std::mem::replace</code>
Need shared ownership	<code>Rc<T></code> (1 thread) / <code>Arc<T></code> (many)
Need shared and mutable	<code>Rc<RefCell<T>></code> / <code>Arc<Mutex<T>></code> (last resort)
Returning data made in the function	return an owned type, not a reference
Counting/grouping into a map	the <code>entry</code> API
Quick prototype, fix later	<code>.clone()</code> — then come back and remove it

The "should I clone?" decision tree

1. Can you **borrow** (`&T` / `&mut T`)? Do that — it is free.
2. Can you **restructure** to avoid needing two owners (split read/write phases, use indices)? Do that.
3. Is the data **small and cheap** (a couple of numbers)? `.clone()` is fine; consider deriving `Copy`.
4. Single-threaded shared ownership? `Rc<T>`.
5. Multi-threaded shared ownership? `Arc<T>`.
6. Shared **and** mutable? `Rc<RefCell<T>>` or `Arc<Mutex<T>>` — and double-check step 2 first.

Four patterns that end most fights

1. Index instead of reference

Storing a `usize` index sidesteps borrow conflicts because a number owns nothing. This is the ECS philosophy in miniature — and it is why `Entity` (Bevy's id) is just a lightweight handle.

```
let selected: usize = 2;           // not &units[2]
apply_buff(&mut units, selected); // re-borrow freshly inside the function
```

2. `split_at_mut` for two parts of one slice

You cannot take `&mut grid[0]` and `&mut grid[1]` at once through indexing, but you can split the slice so the compiler sees two non-overlapping pieces.

```
let (head, tail) = grid.split_at_mut(1);
let row0 = &mut head[0];
let row1 = &mut tail[0];           // both &mut, provably disjoint
```

3. `mem::take` / `mem::replace` to move out of a field

To process a field by value while you still hold `&mut self`, swap it for a default and put the result back. Great for turn queues, command buffers, pending-spawn lists.

```
fn flush_orders(&mut self) {
    let orders = std::mem::take(&mut self.pending); // self.pending is now empty
    for order in orders { self.execute(order); }     // own the orders, no borrow clash
}
```

4. `retain` / `drain` for in-place filtering

```
bullets.retain(|b| b.lifetime > 0.0);           // drop expired bullets in place
let exploded: Vec<_> = mines.drain(..).filter(|m| m.triggered).collect();
```

THE META-LESSON

Almost every borrow-checker fight is the compiler telling you that a *read* and a *write* of the same data overlap in time. Pull them apart — compute what to change first, then change it — and the errors evaporate. This habit also makes your simulation code deterministic and easy to test (Chapter 21).

PART IV · THE BEVY ECS

Bevy ECS Architecture

ECS is the heart of Bevy and the biggest mental shift from Pygame. In Pygame a `Player` is a class that bundles data *and* methods. ECS pulls those apart: **components** are pure data, **systems** are pure functions, and **entities** are just ids that glue components together.

Entities, components, systems

Python – OOP style

```
class Player:
    def __init__(self):
        self.pos = Vec2(0, 0)
        self.hp = 100
        self.sprite = load_image("hero.png")
    def update(self, dt):
        self.pos += self.speed * dt
```

Rust – ECS style

```
#[derive(Component)] struct Position(Vec2);
#[derive(Component)] struct Speed(f32);
#[derive(Component)] struct Health(f32);
#[derive(Component)] struct Player;           // a marker (tag) component, zero data

// Logic lives in a system, not on the data.
fn movement(time: Res<Time>, mut q: Query<(&mut Position, &Speed), With<Player>>()) {
    for (mut pos, speed) in &mut q {
        pos.0 += Vec2::splat(speed.0) * time.delta_secs();
    }
}
```

WHERE DID THE METHODS GO?

They became systems. Instead of `player.update(dt)` you write a `movement` system and the scheduler calls it for *every* entity that has the components it queries. This is what lets Bevy iterate ten thousand entities in a tight, cache-friendly loop.

Required Components: why `spawn(Camera2d)` just works

Since Bevy 0.15 a component can declare that it **requires** others. Spawning `Camera2d` automatically inserts the camera, transform, and projection components it needs. This is why you spawn one thing and get a working camera, and why a `Sprite` brings a `Transform` along.

```
fn setup(mut commands: Commands, assets: Res<AssetServer>) {
    commands.spawn(Camera2d);           // required components fill the rest
    commands.spawn((
        Sprite::from_image(assets.load("hero.png")), // requires Transform → auto-added
        Transform::from_xyz(0.0, 0.0, 0.0),           // we set it explicitly to place it
        Player,
        Health(100.0),
    ));
}
```

```
));
}
```

BUNDLES STILL EXIST, YOU JUST NEED THEM LESS

You can still group components by spawning a tuple `(A, B, C)`. Required components mean you write far fewer of these by hand than older Bevy tutorials show.

Resources: global singletons

A **resource** is one-of-a-kind global data shared by all systems — the score, the current wave number, your minerals and gas. It is the ECS answer to a Python module-level global, but access is tracked so Bevy can schedule systems safely.

```
#[derive(Resource, Default)]
struct Economy { minerals: u32, gas: u32 }

fn spend(mut eco: ResMut<Economy>) { eco.minerals = eco.minerals.saturating_sub(50); }
fn show(eco: Res<Economy>) { info!("minerals: {}", eco.minerals); }
// ^ Res = read-only, ResMut = read-write. Same rules as &/&mut.
```

Messages vs Events — the modern split

Older Bevy (and the draft handbooks) had one concept called "events." Bevy 0.17 split it in two for clarity, and getting this right is the single most important update in this edition.

TWO DIFFERENT THINGS, TWO DIFFERENT TRAITS

Message (`#[derive(Message)]`) — *buffered*. One system writes with a `MessageWriter`, another reads next-ish frame with a `MessageReader`. This is the old `EventWriter` / `EventReader` pattern, re-named. Use it for one-to-many broadcasts processed by systems.

Event (`#[derive(Event)]`) — *observed*. You `trigger` it and an **observer** reacts immediately, often targeted at a specific entity. Covered in Chapter 8.

Most beginner needs — "this system did a thing, that system should know" — are **Messages**.

Rust — a damage Message (write in one system, read in another)

```
#[derive(Message)]
struct DamageDealt { target: Entity, amount: f32 }

// Register it once on the app: app.add_message::();

fn attack(mut writer: MessageWriter<DamageDealt>, q: Query<Entity, With<Enemy>>()) {
    for target in &q {
        writer.write(DamageDealt { target, amount: 10.0 }); // .write, not .send
    }
}

fn apply_damage(mut reader: MessageReader<DamageDealt>, mut hp: Query<&mut Health>) {
    for msg in reader.read() {
        if let Ok(mut health) = hp.get_mut(msg.target) {
            health.0 = (health.0 - msg.amount).max(0.0);
        }
    }
}
```

```
}
}
```

WHY NOT JUST CALL THE FUNCTION?

Decoupling. The `attack` system does not need to know who handles damage, and you can add a second reader (a floating-damage-number system, an audio system) without touching the writer. It is publish/subscribe inside the ECS, like an event bus.

Queries and filters

A `Query` selects entities by the components they have. Filters narrow the selection without fetching extra data:

Filter	Meaning
<code>With<T></code>	entity must have <code>T</code> (but don't fetch it)
<code>Without<T></code>	entity must not have <code>T</code>
<code>Added<T></code>	only entities where <code>T</code> was added this frame
<code>Changed<T></code>	only entities whose <code>T</code> changed since last run (Chapter 10)
<code>Or<(A, B)></code>	combine filters with OR

```
// Living enemies that are not currently stunned.
fn enemy_ai(q: Query<Entity, &Transform>, (With<Enemy>, Without<Stunned>)>) {
    for (e, tf) in &q { /* ... */ }
}
```

GET AND GET_MUT FOR ONE SPECIFIC ENTITY

Iterating a query visits *all* matches. To touch one known entity, use `query.get(entity)` / `query.get_mut(entity)`, which return a `Result`. That is exactly how the `apply_damage` system above reaches the message's `target`.

Commands and recursive despawn

You cannot spawn or despawn entities *while* iterating the world directly — that would invalidate the iteration. `Commands` queues those structural changes to apply at the next sync point.

```
fn cleanup(mut commands: Commands, q: Query<Entity, &Health>) {
    for (entity, hp) in &q {
        if hp.0 <= 0.0 {
            commands.entity(entity).despawn(); // queued; applied after this system
        }
    }
}
```

DESPAWN() NOW TAKES THE CHILDREN WITH IT

Since the relationships rework, `despawn()` is **recursive**: despawning a tank also despawns its turret child entity, and `despawn_recursive` no longer exists. If you want to keep children, re-parent them first. This is the opposite default from very old Bevy, so watch for it when adapting older code.

App, Plugins, States & Schedules

A Bevy game is an `App` you assemble from plugins, organized by states, with systems placed into schedules that run at different times. This chapter is how you keep a growing game from turning into one giant `main` function.

Plugins: your game's modules

A **plugin** bundles related components, resources, messages, and systems and registers them in one place. This is Bevy's answer to a Python module — except registration is explicit and type-driven.

```
fn main() {
    App::new()
        .add_plugins(DefaultPlugins)
        .add_plugins((CombatPlugin, EconomyPlugin, UiPlugin)) // your features
        .init_state::<GameState>()
        .run();
}

pub struct CombatPlugin;
impl Plugin for CombatPlugin {
    fn build(&self, app: &mut App) {
        app.add_message::<DamageDealt>()
            .add_systems(Update, (attack, apply_damage, cleanup).chain());
    }
}
```

SPLIT EARLY

The moment `main` has more than a handful of `add_systems` lines, pull each feature into its own plugin in its own file. Plugins compose: a plugin can add other plugins.

States: menu, playing, paused

A **state** is a top-level mode of the game. Define it as an enum, and Bevy gives you schedules that fire on entering and leaving each state, plus a run condition to gate systems.

```
#[derive(States, Debug, Clone, PartialEq, Eq, Hash, Default)]
enum GameState { #[default] MainMenu, Playing, Paused }

impl Plugin for MenuPlugin {
    fn build(&self, app: &mut App) {
        app
            .add_systems(OnEnter(GameState::MainMenu), spawn_menu) // once, on entry
            .add_systems(OnExit(GameState::MainMenu), despawn_menu) // once, on exit
            .add_systems(Update, menu_buttons.run_if(in_state(GameState::MainMenu)))
            // Gameplay systems only tick while Playing:
            .add_systems(Update, (player_input, enemy_ai)
                .run_if(in_state(GameState::Playing)));
    }
}
```

STATE ENTER/EXIT ↔ SETUP/TEARDOWN

`OnEnter` / `OnExit` are like a context manager's `enter` / `exit` for a whole game mode: spawn the menu UI on enter, despawn it on exit. `run_if(in_state(...))` is the gate that stops gameplay systems from running while you are in the menu.

Ordering systems: sets and `.chain()`

By default Bevy runs systems in parallel in an unspecified order. When one system must run before another (compute damage *before* applying it), you make the order explicit.

```
app.add_systems(Update, (detect_hits, apply_damage, play_sound).chain());
// .chain() = run strictly in this order.

// For looser grouping, name SystemSets and order the sets:
#[derive(SystemSet, Debug, Clone, PartialEq, Eq, Hash)]
enum CombatSet { Detect, Resolve }

app.configure_sets(Update, (CombatSet::Detect, CombatSet::Resolve).chain())
    .add_systems(Update, find_collisions.in_set(CombatSet::Detect))
    .add_systems(Update, (apply_damage, spawn_explosions).in_set(CombatSet::Resolve));
```

A COMMON SILENT BUG

If system B reads data that system A produced *this frame* but you never ordered them, B may run first and miss it — no error, just a one-frame lag or a dropped message. When two systems have a producer/consumer relationship, order them with `.chain()`, `.before()`, or `.after()`.

`Update` vs `FixedUpdate`: simulation needs a fixed step

`Update` runs once per rendered frame, so its `delta_secs()` varies with frame rate. That is fine for visuals but bad for simulation and physics, where variable steps cause nondeterminism and instability. `FixedUpdate` runs at a fixed rate (default 64 Hz), possibly zero or several times per frame, giving you a stable tick — ideal for a city sim or projectile physics.

```
app
    .add_systems(Update, animate_sprites) // visual, per frame
    .add_systems(FixedUpdate, (simulate_economy, integrate_physics));

// Read the fixed step inside FixedUpdate systems:
fn integrate_physics(time: Res<Time<Fixed>>, mut q: Query<(&mut Transform, &Velocity)>) {
    let dt = time.delta_secs(); // constant every tick
    for (mut tf, v) in &mut q { tf.translation += v.0 * dt; }
}
```

RULE OF THUMB

Anything that affects *game logic* and must be reproducible (economy ticks, AI decisions, collision integration) goes in `FixedUpdate`. Anything purely *cosmetic* (sprite animation, camera smoothing, particle fade) goes in `Update`.

| Time & timers

`Timer` and `Time` drive anything that happens "every N seconds": spawning enemy waves, weapon cooldowns, respawn delays.

```
[derive(Resource)]
struct WaveSpawner { timer: Timer }

fn spawn_waves(time: Res<Time>, mut spawner: ResMut<WaveSpawner>, mut commands: Commands) {
    // tick advances the timer; finished() is true on the frame it elapses.
    if spawner.timer.tick(time.delta()).just_finished() {
        commands.spawn((Enemy, Health(50.0) /* , ... */));
    }
}

// Create a repeating 2-second timer:
// WaveSpawner { timer: Timer::from_seconds(2.0, TimerMode::Repeating) }
```

PART IV · THE BEVY ENGINE (CONTINUED)

Observers & Reactive Logic

In Chapter 6 you met **Messages** — the buffered, polled channel you `.read()` every frame. Observers are the other half of Bevy's communication story: **immediate, targeted reactions** that fire the moment something happens. If Messages are like checking your mailbox once a day, observers are like a doorbell.

This is a genuinely new system (stabilised across 0.15–0.18) and it didn't exist in the drafts you started from. It is the idiomatic way to write decoupled gameplay code in modern Bevy.

Two kinds of events

Since 0.17, the word `Event` is reserved for the observer system, while the old buffered type was renamed `Message`. Keep them straight:

Concept	Buffered (Message)	Observed (Event)
Derive	<code>#[derive(Message)]</code>	<code>#[derive(Event)]</code> or <code>#[derive(EntityEvent)]</code>
Send	<code>MessageWriter::write</code>	<code>commands.trigger(...)</code>
Receive	<code>MessageReader::read</code> in a system	observer fn <code>On<E></code>
Timing	next time the reader runs	immediately / at command flush
Target	global stream	global or a specific entity

FOR THE PYTHON DEVELOPER

An `Event` + observer is essentially the **observer pattern** or a signal/slot system (think Qt signals, or `blinker` in Flask). You `trigger` a signal and any number of connected callbacks run. Bevy just wires it through the ECS so your callbacks are ordinary systems with full query access.

A global observer

An observer is a function whose first parameter is `On<YourEvent>`. Register it once.

Rust – a global observer reacting to a game-wide event

```
use bevy::prelude::*;

#[derive(Event)]
struct PlayerLeveledUp { new_level: u32 }

fn on_level_up(trigger: On<PlayerLeveledUp>, mut commands: Commands) {
    let level = trigger.event().new_level;
    info!("Ding! You are now level {level}");
    // spawn some particles, play a sound, etc.
}

fn main() {
    App::new()
        .add_plugins(DefaultPlugins)
        .add_observer(on_level_up) // <- register the observer
        .run();
}
```



```

}

// Somewhere in a system, fire it:
fn award_xp(mut commands: Commands /* , ... */) {
    commands.trigger(PlayerLeveledUp { new_level: 7 });
}

```

`commands.trigger(...)` is **deferred** — it runs when commands flush (end of the system). If you hold a `&mut World` directly you can `world.trigger(...)` for an **immediate** call.

Entity-targeted events

The real power is targeting a *specific* entity. Derive `EntityEvent` and trigger it on an entity; the observer reads `trigger.entity` to know who was hit.

Rust — decoupled damage → death reaction

```

#[derive(EntityEvent)]
struct Damage { amount: f32 }

#[derive(Component)]
struct Health(f32);

// Observe Damage on whichever entity it targets.
fn apply_damage(
    trigger: On<Damage>,
    mut healths: Query<&mut Health>,
    mut commands: Commands,
) {
    let entity = trigger.entity;
    if let Ok(mut hp) = healths.get_mut(entity) {
        hp.0 -= trigger.event().amount;
        if hp.0 <= 0.0 {
            commands.entity(entity).despawn(); // recursive in 0.18
        }
    }
}

// Fire it at a particular enemy:
fn resolve_hit(mut commands: Commands, enemy: Entity) {
    commands.trigger(Damage { amount: 25.0 }); // see note on targeting below
}

```

TRIGGERING ON AN ENTITY

The exact call to target an entity is `commands.trigger(event)` where the event itself carries the target (`EntityEvent` derives an entity field), or via `commands.entity(e).trigger(event)` style depending on your event shape. The pattern that matters: the observer recovers the target through `trigger.entity` and never has to be told *which* system produced the hit. The shooter, the trap, and the poison tick all just `trigger(Damage{..})`.

Per-entity observers

You can also attach an observer to **one entity only** with `.observe(...)` at spawn time — perfect for a button, a trap tile, or a boss with unique behaviour.

```
commands
    .spawn((Trap, Transform::default()))
    .observe(|trigger: On<Stepped>, mut commands: Commands| {
        // only THIS trap reacts here
        commands.trigger(Damage { amount: 10.0 });
    });
```

Lifecycle hooks: react to the ECS itself

Bevy emits built-in events when components are added or removed. Observe them to run setup or teardown logic automatically — no manual bookkeeping.

Event	Fires when...
<code>On<Add, C></code>	component <code>C</code> is first added to an entity
<code>On<Insert, C></code>	<code>C</code> is added <i>or overwritten</i>
<code>On<Replace, C></code>	the old value of <code>C</code> is about to be replaced/removed
<code>On<Remove, C></code>	<code>C</code> is removed (or the entity despawns)

Rust — auto-register an enemy in a spatial index the instant it spawns

```
fn on_enemy_added(trigger: On<Add, Enemy>, mut index: ResMut<SpatialIndex>) {
    index.insert(trigger.entity);
}

fn on_enemy_removed(trigger: On<Remove, Enemy>, mut index: ResMut<SpatialIndex>) {
    index.remove(trigger.entity);
}

app.add_observer(on_enemy_added)
    .add_observer(on_enemy_removed);
```

WHEN TO USE WHICH

Reach for an **observer/Event** when a reaction should happen *now* and is naturally one-to-one or one-to-few (this bullet hit this ship). Reach for a **Message** when many producers feed many consumers and a one-frame delay is fine (input actions, UI events, analytics). Both are first-class; mixing them is normal.

What we covered

You can now fire immediate, targeted reactions, attach behaviour to individual entities, and hook into the lifecycle of components themselves. Combined with Messages from Chapter 6, you have the full vocabulary for decoupled game logic — which the genre workshops in Part VI lean on heavily for production queues, deaths, and pickups.

Assets, Sprites & Input

A game needs to load pictures and read the keyboard. This chapter wires up the plumbing every genre in Part VI relies on: the asset server, sprites and a camera, keyboard and mouse input, and turning a mouse click into a world position.

Loading assets

The `AssetServer` is a resource. You ask it for a path and it hands back a `Handle<T>` — a cheap, clonable ticket. Loading is asynchronous; the handle is valid immediately and the pixels stream in on a background thread.

Rust – load a texture and spawn a sprite

```
use bevy::prelude::*;

fn setup(mut commands: Commands, asset_server: Res<AssetServer>) {
    // A 2D camera. Camera2d pulls in everything it needs via Required Components.
    commands.spawn(Camera2d);

    let texture: Handle<Image> = asset_server.load("sprites/player.png");
    commands.spawn((
        Sprite::from_image(texture),
        Transform::from_xyz(0.0, 0.0, 0.0),
        Player,
    ));
}
```

FOR THE PYTHON DEVELOPER

A `Handle<Image>` is like a key into a global dict of loaded assets — not the bytes themselves. Cloning a handle is like copying the key, not the file. This is how Rust lets a thousand bullets share one texture with zero copying: they all hold the same handle.

ASSET PATHS LIVE IN `ASSETS/`

Paths are relative to an `assets/` folder next to your executable.

`asset_server.load("sprites/player.png")` looks for `assets/sprites/player.png`. A missing file fails *gracefully* (you get a broken handle and a log warning), so a black screen usually means a wrong path, not a crash.

Sprites and the camera

In 0.18, **Required Components** mean you spawn the meaningful thing and Bevy fills in the rest.

`Camera2d` brings its own `Transform`, `Camera`, projection, and so on. `Sprite` requires a `Transform`, so spawn them together.

```
// Tint, flip, and size a sprite without separate components:
commands.spawn((
    Sprite {
        image: asset_server.load("sprites/enemy.png"),
        color: Color::srgb(1.0, 0.6, 0.6), // red-ish tint
        flip_x: true,
```

```

        ..default()
    },
    Transform::from_xyz(120.0, 0.0, 0.0),
));

```

Keyboard input

Keyboard and mouse-button state live in the `ButtonInput<T>` resource. Three questions you can ask each frame:

Method	True when...	Use for
<code>pressed(key)</code>	the key is held down	continuous movement
<code>just_pressed(key)</code>	it went down <i>this frame</i>	jump, fire, confirm
<code>just_released(key)</code>	it came up <i>this frame</i>	charge-and-release

Rust – WASD movement + a one-shot fire key

```

fn move_player(
    keys: Res<ButtonInput<KeyCode>>,
    time: Res<Time>,
    mut q: Query<&mut Transform, With<Player>>,
) -> Result {
    let mut tf = q.single_mut()?; // single_mut() returns Result in 0.18
    let speed = 300.0 * time.delta_secs();

    let mut dir = Vec2::ZERO;
    if keys.pressed(KeyCode::KeyW) { dir.y += 1.0; }
    if keys.pressed(KeyCode::KeyS) { dir.y -= 1.0; }
    if keys.pressed(KeyCode::KeyA) { dir.x -= 1.0; }
    if keys.pressed(KeyCode::KeyD) { dir.x += 1.0; }

    tf.translation += (dir.normalize_or_zero() * speed).extend(0.0);

    if keys.just_pressed(KeyCode::Space) {
        info!("pew!");
    }
    Ok(())
}

```

SYSTEMS CAN RETURN `RESULT`

Returning `Result` lets you use `?` on `single_mut()` and other fallible calls. If the query is empty (no player yet, or two players) the system bails cleanly that frame instead of panicking. This is a major 0.18 ergonomics win over the old `.single()`-panics-on-fail behaviour.

Mouse buttons and motion

```

fn shoot(buttons: Res<ButtonInput<MouseButton>>) {
    if buttons.just_pressed(MouseButton::Left) {
        // fire
    }
}

```

For raw motion and scroll you read **messages** (Chapter 6), not a resource:

```
use bevy::input::mouse::{MouseMotion, MouseWheel};

fn look(mut motion: MessageReader<MouseMotion>, mut wheel: MessageReader<MouseWheel>) {
    for ev in motion.read() {
        // ev.delta is a Vec2 of how far the mouse moved this frame
    }
    for ev in wheel.read() {
        // ev.y is scroll amount – zoom the camera, scroll a list
    }
}
```

Click → world position

This is the single most-asked 2D question: *where in the game world did the player click?* A click gives you a **screen** pixel; you must convert through the camera.

Rust – cursor to world coordinates (point-and-click, RTS selection)

```
fn cursor_to_world(
    windows: Query<&Window>,
    camera_q: Query<&Camera, &GlobalTransform>,
) -> Result {
    let window = windows.single()?;
    let (camera, cam_tf) = camera_q.single()?;

    if let Some(cursor) = window.cursor_position() {
        // viewport_to_world_2d returns Result in 0.18
        if let Ok(world) = camera.viewport_to_world_2d(cam_tf, cursor) {
            info!("clicked world point: {world:?}");
        }
    }
    Ok(())
}
```

Y IS UP IN WORLD SPACE, DOWN IN SCREEN SPACE

Bevy's world has +Y pointing up; window cursor coordinates have +Y pointing down. `viewport_to_world_2d` handles the flip for you — one more reason never to do the maths by hand.

A first taste of UI

Bevy's UI is just more entities with `Node` components, laid out with a flexbox-like model. You'll use it for HUDs, menus, and the card/visual-novel genres.

```
fn spawn_hud(mut commands: Commands) {
    commands.spawn((
        Text::new("Score: 0"),
        Node {
            position_type: PositionType::Absolute,
            top: Val::Px(12.0),
            left: Val::Px(12.0),
            ..default()
        },
    ));
}
```

```
    ));  
}
```

| What we covered

You can load textures, put sprites and a camera on screen, read every flavour of keyboard and mouse input, convert a click into a world position, and stamp text onto a HUD. That's the full input/output surface the genre workshops need. Next we make it *fast*.

Performance & Change Detection

Rust is fast by default, but a game ships thousands of entities updating sixty times a second. This chapter is about *not* doing unnecessary work: only reacting to what changed, laying data out for the cache, avoiding per-frame allocations, and pooling. We'll frame it around a SimCity-style sim where tens of thousands of tiles must stay cheap.

Change detection: do nothing when nothing changed

Every component query can be filtered by *whether the data changed*. Bevy tracks this for you.

Filter	Matches entities whose component...
<code>Changed<C></code>	was mutated (or added) since this system last ran
<code>Added<C></code>	was added since this system last ran

Rust – only re-render a tile when its zoning actually changes

```
fn recolor_changed_tiles(
    mut q: Query<(&Zone, &mut Sprite), Changed<Zone>>, // <- the magic filter
) {
    for (zone, mut sprite) in &mut q {
        sprite.color = zone.display_color();
    }
}
```

On a 200×200 city that's 40,000 tiles. Without the filter you recolour all of them every frame. With it, you touch only the handful the player just edited. Same idea powers "dirty flag" UI updates, recomputing a nav grid only when obstacles move, and so on.

MUTABLE ACCESS MARKS AS CHANGED — EVEN IF YOU DON'T CHANGE ANYTHING

Iterating `&mut Sprite` flags every matched sprite as changed, which can cascade into *other* systems' `Changed` filters. If you only sometimes write, check first: `if sprite.color != target { sprite.color = target; }`. Reading through `&mut` and not writing still trips the flag.

FOR THE PYTHON DEVELOPER

There's no NumPy here, but the spirit is the same: avoid Python-style per-element loops over everything each frame. Change detection is how you get "only process the diff" without writing the bookkeeping yourself.

Data layout: structure-of-arrays for free

Bevy's ECS stores components of the same type contiguously. Iterating a `Query<&Position>` walks a tight array — cache-friendly by construction. You get **structure-of-arrays (SoA)** performance while writing what looks like object code.

The practical lesson: **keep hot components small and split them**. A tile that's iterated for rendering shouldn't also carry a 200-byte `History` blob in the same component — that blows the cache line. Put cold data in its own component and query it only when needed.

```
// Hot: touched every frame. Keep it tiny.
#[derive(Component)] struct Position(Vec2);
#[derive(Component)] struct Zone(ZoneKind);

// Cold: touched rarely. Separate component, separate array.
#[derive(Component)] struct TileHistory { events: Vec<TileEvent> }
```

Avoiding per-frame allocations

Allocating (`Vec::new`, `String`, boxing) inside a hot loop is the classic frame-time killer. Two habits fix most of it.

Reserve once, reuse:

```
// BAD: a fresh Vec every frame, grown element by element.
fn collect_targets_bad(q: Query<&Position, With<Enemy>>()) {
    let mut targets = Vec::new();
    for p in &q { targets.push(p.0); }
    // ...
}

// GOOD: keep the buffer in a resource; clear() keeps the capacity.
#[derive(Resource, Default)]
struct TargetBuffer(Vec<Vec2>);

fn collect_targets(q: Query<&Position, With<Enemy>>(), mut buf: ResMut<TargetBuffer>) {
    buf.0.clear(); // length 0, capacity retained – no realloc
    buf.0.extend(q.iter().map(|p| p.0));
}
```

`clear()` does **not** free the backing memory, so next frame's `extend` reuses it. `Vec::with_capacity(n)` up front avoids the doubling reallocations entirely when you know `n`.

Object pooling: bullets that never deallocate

A shoot'em up spawns and despawns bullets constantly. Spawning/despawning ECS entities is cheap but not free, and recreating textures or buffers each time is wasteful. The **pool** pattern keeps a fixed set of entities and toggles them active/inactive.

Rust – a minimal bullet pool

```
#[derive(Component)] struct Bullet { active: bool, vel: Vec2 }

// Spawn the whole pool once, inactive and hidden.
fn init_pool(mut commands: Commands) {
    for _ in 0..512 {
        commands.spawn((
            Bullet { active: false, vel: Vec2::ZERO },
            Transform::default(),
            Visibility::Hidden,
        ));
    }
}

// "Fire" = find an inactive bullet and switch it on. No spawn, no allocation.
fn fire(
```



```

    mut q: Query<(&mut Bullet, &mut Transform, &mut Visibility)>,
    muzzle: Vec2,
    dir: Vec2,
) {
    if let Some((mut b, mut tf, mut vis)) =
        q.iter_mut().find(|(b, _, _)| !b.active)
    {
        b.active = true;
        b.vel = dir * 600.0;
        tf.translation = muzzle.extend(0.0);
        *vis = Visibility::Visible;
    }
}

```

When a bullet leaves the screen you set `active = false` and `Visibility::Hidden` instead of despawning. The entity is recycled forever.

Parallelism: `par_iter_mut`

For genuinely heavy per-entity work (thousands of agents doing independent maths), Bevy can run the loop across threads with one method swap — *if* the bodies don't touch shared state.

```

fn integrate(mut q: Query<(&mut Transform, &Velocity)>, time: Res<Time>) {
    let dt = time.delta_secs();
    q.par_iter_mut().for_each(|(mut tf, v)| {
        tf.translation += v.0 * dt; // independent per entity – safe to parallelise
    });
}

```

MEASURE BEFORE YOU PARALLELISE

Threading has overhead. For a few hundred cheap entities, plain `&mut q` iteration wins. Reach for `par_iter_mut` only when a single system is measurably eating your frame and its work is embarrassingly parallel. Chapter 20 shows how to time a system.

DON'T PARALLELISE SYSTEMS THAT NEED `COMMANDS` PER ITEM OR SHARED RESOURCES

`par_iter_mut` bodies must be self-contained. Spawning, despawning, or writing a shared `ResMut` from inside the closure is a conflict. Collect results first, apply them after.

What we covered

Change detection to skip untouched work, small hot components for cache-friendly SoA iteration, buffer reuse and `with_capacity` to kill allocations, a bullet pool, and one-line parallel iteration. These are the levers that keep a busy 90s-style sim running at sixty frames. Now we turn to the design patterns that keep the *code* maintainable.

PART V · IDIOMATIC PATTERNS FOR GAMES

Design Patterns in Rust

The Gang-of-Four patterns you may know from Python translate to Rust — but Rust's ownership model and enums often give you a cleaner version, and sometimes the ECS makes the pattern disappear entirely. This chapter adapts the community *Rust Design Patterns* catalogue to game code, and adds two patterns Rust gives you for free: **RAII guards** and the **type-state pattern**.

Builder — readable configuration

When a thing has many optional settings, a builder beats a constructor with ten arguments.

Python — keyword args make this a non-problem

```
level = Level(width=64, height=64, seed=42, fog=True)
```

Rust — a builder recreates that ergonomics

```
#[derive(Default)]
struct LevelConfig { width: u32, height: u32, seed: u64, fog: bool }

struct LevelBuilder(LevelConfig);

impl LevelBuilder {
    fn new() -> Self { Self(LevelConfig { width: 64, height: 64, ..default() }) }
    fn size(mut self, w: u32, h: u32) -> Self { self.0.width = w; self.0.height = h; self }
    fn seed(mut self, s: u64) -> Self { self.0.seed = s; self }
    fn fog(mut self, on: bool) -> Self { self.0.fog = on; self }
    fn build(self) -> LevelConfig { self.0 }
}

let cfg = LevelBuilder::new().size(128, 128).seed(42).fog(true).build();
```

Each method takes `self` by value and returns it, so calls chain. This is exactly how you configure a Bevy `App` or a `PluginGroup`.

Strategy — three ways, pick the Rust-iest

"Swap the algorithm at runtime" (different AI behaviours) has three idiomatic forms. Knowing which to choose is the lesson.

1. Enum dispatch — best when the set of strategies is fixed and known:

```
#[derive(Component)]
enum AiBehavior { Aggressive, Defensive, Patrol { radius: f32 } }

fn run_ai(mut q: Query<(&AiBehavior, &mut Transform)>) {
    for (behavior, mut tf) in &mut q {
        match behavior {
            AiBehavior::Aggressive => { /* charge player */ }
            AiBehavior::Defensive  => { /* hold ground */ }
            AiBehavior::Patrol { radius } => { /* circle within *radius */ }
        }
    }
}
```

```
}
}
```

No heap allocation, no dynamic dispatch, exhaustively checked by the compiler. **Prefer this.**

2. Trait objects — when strategies are open-ended or plugin-supplied:

```
trait Behavior: Send + Sync { fn decide(&self, ctx: &Ctx) -> Action; }

#[derive(Component)]
struct Brain(Box<dyn Behavior>); // pay one pointer indirection for flexibility
```

3. ECS-idiomatic — make the strategy a *marker component* and write one system per behaviour, each with a `With<Tag>` filter. Most "Bevy-native" and parallelises for free:

```
#[derive(Component)] struct Aggressive;
#[derive(Component)] struct Defensive;

fn aggressive_ai(q: Query<&mut Transform, With<Aggressive>>) { /* ... */ }
fn defensive_ai(q: Query<&mut Transform, With<Defensive>>) { /* ... */ }
```

DECISION RULE

Fixed, small set → **enum**. Needs to be extended by code you don't control → **trait object**. Already living in the ECS and you want parallelism → **marker components**.

Command — undo/redo and replays

Represent an action as data, not a function call. Store it; you can log it, replay it, or reverse it. Card games and strategy games live on this.

Rust — reversible game commands

```
enum GameCommand {
    MoveUnit { unit: Entity, from: Vec2, to: Vec2 },
    SpendGold { amount: u32 },
}

impl GameCommand {
    fn undo(&self) -> GameCommand {
        match self {
            GameCommand::MoveUnit { unit, from, to } =>
                GameCommand::MoveUnit { unit: *unit, from: *to, to: *from },
            GameCommand::SpendGold { amount } =>
                GameCommand::SpendGold { amount: 0 }, // refund handled by caller
        }
    }
}

#[derive(Resource, Default)]
struct History { done: Vec<GameCommand> } // push on apply, pop on undo
```

Because the command is a plain enum, you can `serde`-serialise the whole `done` stack to a file and get **replays** for free (Chapter 20).

Newtype — make illegal states unrepresentable

A bare `u32` for health and a bare `u32` for gold are interchangeable to the compiler, so you *will* eventually subtract gold from health. Wrap them.

Rust — the newtype pattern for unit safety

```
#[derive(Component, Clone, Copy)] struct Hp(pub f32);
#[derive(Component, Clone, Copy)] struct Mana(pub f32);
#[derive(Resource, Clone, Copy)] struct Gold(pub u32);

fn heal(hp: &mut Hp, amount: f32) { hp.0 += amount; }
// heal(&mut mana, 10.0); // <- compile error: Mana is not Hp. Bug caught at build time.
```

FOR THE PYTHON DEVELOPER

Python would let `hp + gold` run and produce a nonsense number you'd debug at 2 a.m. Newtypes push that class of bug from runtime to compile time — the recurring theme of this whole book.

State machine — enums as states

A weapon is `Ready`, `Firing`, `Reloading`, or `Jammed`. Model it as an enum and transition in a `match`. The compiler forces you to handle every state.

```
#[derive(Component)]
enum WeaponState { Ready, Firing { frames_left: u8 }, Reloading { frames_left: u8 } }

fn tick_weapon(mut q: Query<&mut WeaponState>) {
    for mut w in &mut q {
        *w = match *w {
            WeaponState::Firing { frames_left: 0 } => WeaponState::Reloading { frames_left:
30 },
            WeaponState::Firing { frames_left }      => WeaponState::Firing { frames_left: fram
es_left - 1 },
            WeaponState::Reloading { frames_left: 0 } => WeaponState::Ready,
            WeaponState::Reloading { frames_left }    => WeaponState::Reloading {
frames_left: frames_left - 1 },
            WeaponState::Ready => WeaponState::Ready,
        };
    }
}
```

Default — sensible zero values

Derive `Default` so a struct has an obvious starting state; combine with struct-update syntax (`..default()`) to override only what differs. You've seen this all over Bevy already.

```
#[derive(Default)]
struct Settings { volume: f32, fullscreen: bool, difficulty: Difficulty }

let s = Settings { volume: 0.8, ..default() };
```

RAII guards — cleanup that can't be forgotten

This pattern barely exists in Python (you reach for `with`/context managers). In Rust, a value runs its `Drop` code automatically when it goes out of scope — so you bind cleanup to a value's lifetime. A *guard* is a value whose only job is that teardown.

Rust — a scoped profiling guard

```
struct ScopeTimer { label: &'static str, start: std::time::Instant }
```

```
impl ScopeTimer {
    fn new(label: &'static str) -> Self { Self { label, start: std::time::Instant::now() } }
```

```
impl Drop for ScopeTimer {
    fn drop(&mut self) {
        info!("{ } took { }", self.label, self.start.elapsed());
    }
}
```

```
fn heavy_system() {
    let _t = ScopeTimer::new("pathfinding"); // timer starts
    // ... do work ...
}
```

// _t dropped here → logs automatically

DROP IS DETERMINISTIC

Unlike a Python `del` (which fires whenever the garbage collector feels like it), Rust guarantees `drop` runs at the exact moment the value leaves scope. That's why RAII is reliable enough for locks, file handles, and GPU resources.

Type-state — encode state in the type

The most advanced pattern here, and a Rust speciality: make *invalid sequences of calls* fail to compile by giving each state its own type. A classic example: a connection you must `build()` before you can `send()`.

Rust — a level that must be validated before it can be played

```
struct Draft; // marker types – zero size, exist only at the type level
struct Validated;

struct Level<State> { tiles: Vec<Tile>, _state: std::marker::PhantomData<State> }
```

```
impl Level<Draft> {
    fn new() -> Self { Self { tiles: vec![], _state: std::marker::PhantomData } }
    fn add_tile(mut self, t: Tile) -> Self { self.tiles.push(t); self }
    fn validate(self) -> Result<Level<Validated>, String> {
        if self.tiles.is_empty() { return Err("empty level".into()); }
        Ok(Level { tiles: self.tiles, _state: std::marker::PhantomData })
    }
}
```

```
impl Level<Validated> {
    fn play(&self) { /* only callable on a validated level */ }
```

```
// let l = Level::new().add_tile(t);  
// l.play();           // <- compile error: play() doesn't exist on Level<Draft>  
// l.validate()?.play(); // OK
```

`PhantomData` carries the `State` type without storing any data. The payoff: "you forgot to validate" becomes a compiler error, not a crash. Use it sparingly — for genuinely important sequencing — because it adds ceremony.

| What we covered

Builder for configuration, Strategy in three flavours (and when each wins), Command for undo/replay, Newtype and state-enums to make bad states unrepresentable, Default for ergonomics, plus the two Rust-native patterns — RAII `Drop` guards and the type-state pattern. With the language, the engine, and the patterns in hand, Part VI puts them to work building actual 90s games.

PART VI · GENRE WORKSHOPS — CLONING THE 90S

Workshop: Shoot'em Up

The inspiration: *R-Type* (1987) and *Gradius* — side-scrolling, wave after wave of enemies, a hail of bullets, tight collision. The shmup is the perfect first workshop because it exercises spawning, movement, the bullet pool from Chapter 10, and collision detection without much state.

Everything here is wired through a single `ShmupPlugin` so it drops into a real project.

The plugin skeleton

```
use bevy::prelude::*;

pub struct ShmupPlugin;

impl Plugin for ShmupPlugin {
    fn build(&self, app: &mut App) {
        app
            .insert_resource(WaveTimer(Timer::from_seconds(1.5, TimerMode::Repeating)))
            .add_systems(Startup, spawn_player)
            .add_systems(Update, (
                player_input,
                spawn_waves,
                move_bullets,
                move_enemies,
                bullet_hits_enemy,
                despawn_offscreen,
            ));
    }
}
```

Components and the player

```
#[derive(Component)] struct Player;
#[derive(Component)] struct Enemy { hp: f32 }
#[derive(Component)] struct Bullet;
#[derive(Component)] struct Velocity(Vec2);

fn spawn_player(mut commands: Commands, assets: Res<AssetServer>) {
    commands.spawn(Camera2d);
    commands.spawn((
        Player,
        Sprite::from_image(assets.load("ship.png")),
        Transform::from_xyz(-300.0, 0.0, 0.0),
    ));
}
```

Firing bullets

```
fn player_input(
    keys: Res<ButtonInput<KeyCode>>,
    time: Res<Time>,
    ...)
```

```

    mut commands: Commands,
    mut q: Query<&mut Transform, With<Player>>,
) -> Result {
    let mut tf = q.single_mut()?;
    let speed = 320.0 * time.delta_secs();
    if keys.pressed(KeyCode::ArrowUp) { tf.translation.y += speed; }
    if keys.pressed(KeyCode::ArrowDown) { tf.translation.y -= speed; }

    if keys.just_pressed(KeyCode::Space) {
        commands.spawn((
            Bullet,
            Velocity(Vec2::new(700.0, 0.0)),
            Transform::from_translation(tf.translation),
            Sprite { color: Color::srgb(1.0, 0.9, 0.2), custom_size: Some(Vec2::new(8.0, 3.0))
        , ..default() },
        ));
    }
    Ok(())
}

```

POOL IT FOR THE REAL THING

Spawning a bullet per shot is fine to start. When the screen fills with hundreds, switch to the bullet pool from Chapter 10 — same `Velocity` / `Transform`, but recycle inactive entities instead of spawning.

Enemy waves

```

#[derive(Resource)] struct WaveTimer(Timer);

fn spawn_waves(
    time: Res<Time>,
    mut timer: ResMut<WaveTimer>,
    mut commands: Commands,
    assets: Res<AssetServer>,
) {
    if timer.0.tick(time.delta()).just_finished() {
        for i in 0..5 {
            let y = 200.0 - i as f32 * 100.0;
            commands.spawn((
                Enemy { hp: 30.0 },
                Velocity(Vec2::new(-120.0, 0.0)),
                Transform::from_xyz(400.0, y, 0.0),
                Sprite::from_image(assets.load("enemy.png")),
            ));
        }
    }
}

```

Movement (shared by bullets and enemies)

```

fn move_bullets(time: Res<Time>, mut q: Query<(&mut Transform, &Velocity), With<Bullet>>) {
    let dt = time.delta_secs();
    for (mut tf, v) in &mut q { tf.translation += v.0.extend(0.0) * dt; }
}

```



```
fn move_enemies(time: Res<Time>, mut q: Query<(&mut Transform, &Velocity), With<Enemy>>) {
    let dt = time.delta_secs();
    for (mut tf, v) in &mut q { tf.translation += v.0.extend(0.0) * dt; }
}
```

Collision: bullet hits enemy

A shmup needs only cheap circle/AABB overlap. Here's a simple distance check.

Rust – `0(bullets × enemies)` is fine for a shmup; spatial grids come later

```
fn bullet_hits_enemy(
    mut commands: Commands,
    bullets: Query<Entity, &Transform>, With<Bullet>>,
    mut enemies: Query<Entity, &Transform, &mut Enemy>,
) {
    for (be, bt) in &bullets {
        for (ee, et, mut enemy) in &mut enemies {
            if bt.translation.distance(et.translation) < 24.0 {
                enemy.hp -= 10.0;
                commands.entity(be).despawn();           // bullet gone
                if enemy.hp <= 0.0 {
                    commands.entity(ee).despawn();       // enemy gone (recursive)
                }
                break;                                     // this bullet is spent
            }
        }
    }
}
```

BORROW-CHECKER CORNER YOU'LL ACTUALLY HIT

You cannot despawn while iterating *and* keep mutating the same query — that's why we use `Commands` (deferred) rather than touching the `World` directly. The `break` after a hit also prevents one bullet from double-counting against two overlapping enemies in the same frame.

Cleanup

```
fn despawn_offscreen(mut commands: Commands, q: Query<Entity, &Transform>) {
    for (e, tf) in &q {
        if tf.translation.x.abs() > 900.0 || tf.translation.y.abs() > 600.0 {
            commands.entity(e).despawn();
        }
    }
}
```

What you built

A self-contained side-scrolling shooter: a controllable ship, timed enemy waves, bullets, distance-based collision with damage and death, and offscreen cleanup — all behind one plugin. The patterns here (spawn, move via `Velocity`, collide, despawn) recur in every action game. Next we tackle the genre that defined 90s PC gaming: the real-time strategy game.

Workshop: Strategy — Real-Time & Turn-Based

The inspiration: the two pillars of 90s strategy. *Command & Conquer* (1995) and *Dune II* for **real-time strategy (RTS)** — select with a drag-box, right-click to order, harvest, queue production. And *Civilization* (1991) and *X-COM* (1994) for **turn-based strategy (STR/TBS)** — a turn cycle, action points, tile-by-tile movement. They share a worldview (units on a map, an economy, orders) but differ in *time*: one simulates continuously, the other waits for you. We build the real-time core first, then convert the same units to a turn-based model so you can feel exactly where the genres diverge. It combines the click-to-world maths from Chapter 9, a deterministic `FixedUpdate` simulation from Chapter 7, **Messages** for production requests, and **Observers** for unit-spawned reactions.

FOR THE PYTHON DEVELOPER

If you've built a grid sim in Python with a big `update()` loop, the shift here is splitting that loop into many small systems that the scheduler runs for you — and letting the type system, not your discipline, keep selection state and orders coherent.

Plugin and core components

```
use bevy::prelude::*;

pub struct RtsPlugin;

impl Plugin for RtsPlugin {
    fn build(&self, app: &mut App) {
        app
            .insert_resource(Credits(1000))
            .add_message::<BuildRequest>()           // production channel
            .add_observer(on_unit_spawned)          // react to new units
            .add_systems(Startup, setup)
            .add_systems(Update, (
                begin_drag_select,
                issue_move_order,
                enqueue_production,
                process_build_requests,
            ))
            .add_systems(FixedUpdate, (move_units, harvest_credits));
    }
}

#[derive(Component)] struct Unit;
#[derive(Component)] struct Selected;
#[derive(Component)] struct MoveOrder(Vec2);
#[derive(Component)] struct Harvester { rate: u32 }
#[derive(Resource)] struct Credits(u32);
```

Selection: click to world, then mark entities

```
fn begin_drag_select(
    buttons: Res<ButtonInput<MouseButton>>,
    windows: Query<&Window>,
```

```

cameras: Query<(&Camera, &GlobalTransform)>,
units: Query<Entity, &Transform>, With<Unit>>,
mut commands: Commands,
) -> Result {
    if !buttons.just_pressed(MouseButton::Left) { return Ok(()); }

    let window = windows.single()?;
    let (camera, cam_tf) = cameras.single()?;
    let Some(cursor) = window.cursor_position() else { return Ok(()); };
    let Ok(world) = camera.viewport_to_world_2d(cam_tf, cursor) else { return Ok(()); };

    // Single-click select: pick the nearest unit within a radius.
    for (e, tf) in &units {
        let hit = tf.translation.truncate().distance(world) < 24.0;
        if hit { commands.entity(e).insert(Selected); }
        else { commands.entity(e).remove::(); }
    }
    Ok(())
}

```

A REAL DRAG-BOX

To select a rectangle, record the world point on `just_pressed` and the point on `just_released`, then mark every unit whose position falls between the two corners. The single-click version above keeps the workshop focused; the extension is a few lines.

Orders: right-click sets a destination

```

fn issue_move_order(
    buttons: Res<ButtonInput<MouseButton>>,
    windows: Query<&Window>,
    cameras: Query<(&Camera, &GlobalTransform)>,
    selected: Query<Entity, With<Selected>>,
    mut commands: Commands,
) -> Result {
    if !buttons.just_pressed(MouseButton::Right) { return Ok(()); }
    let window = windows.single()?;
    let (camera, cam_tf) = cameras.single()?;
    let Some(cursor) = window.cursor_position() else { return Ok(()); };
    let Ok(world) = camera.viewport_to_world_2d(cam_tf, cursor) else { return Ok(()); };

    for e in &selected {
        commands.entity(e).insert(MoveOrder(world)); // attach a destination
    }
    Ok(())
}

```

Deterministic movement in FixedUpdate

Unit movement is *game logic*, so it belongs in `FixedUpdate` for a stable, reproducible simulation (Chapter 7). When a unit reaches its order, the order component is removed.

```

fn move_units(
    time: Res<Time<Fixed>>,
    mut commands: Commands,

```

```

    mut q: Query<(Entity, &mut Transform, &MoveOrder)>,
) {
    let step = 80.0 * time.delta_secs(); // constant per tick
    for (e, mut tf, order) in &mut q {
        let pos = tf.translation.truncate();
        let to = order.0 - pos;
        if to.length() <= step {
            tf.translation = order.0.extend(tf.translation.z);
            commands.entity(e).remove::<MoveOrder>(); // arrived
        } else {
            tf.translation += (to.normalize() * step).extend(0.0);
        }
    }
}

```

Economy: harvesters tick credits

```

fn harvest_credits(mut credits: ResMut<Credits>, q: Query<&Harvester>) {
    for h in &q { credits.0 += h.rate; } // one tick = one harvest pulse
}

```

Production: a Message channel + an Observer

A factory enqueues a **BuildRequest** message; a system drains the queue, checks credits, and spawns the unit. The spawn fires an **observer** so anything else (sound, minimap ping) reacts without the factory knowing about it.

Rust – Messages for the request, Observer for the reaction

```

#[derive(Message)] struct BuildRequest { cost: u32 }
#[derive(Event)] struct UnitSpawned; // observer event

fn enqueue_production(
    keys: Res<ButtonInput<KeyCode>>,
    mut writer: MessageWriter<BuildRequest>,
) {
    if keys.just_pressed(KeyCode::KeyB) {
        writer.write(BuildRequest { cost: 300 });
    }
}

fn process_build_requests(
    mut reader: MessageReader<BuildRequest>,
    mut credits: ResMut<Credits>,
    mut commands: Commands,
) {
    for req in reader.read() {
        if credits.0 >= req.cost {
            credits.0 -= req.cost;
            commands
                .spawn((Unit, Transform::from_xyz(0.0, 0.0, 0.0)))
                .observe(|_t: On<UnitSpawned>| info!("unit ready!"));
            // fire the reaction
            commands.trigger(UnitSpawned);
        } else {
            info!("not enough credits");
        }
    }
}

```

```

    }
}

fn on_unit_spawned(_t: On<UnitSpawned>) {
    // global reaction: e.g. ping the minimap
}

```

WHY BOTH MECHANISMS?

The **message** decouples *who asks* to build from *who fulfils* it and naturally batches many requests per frame. The **observer** decouples *the moment of spawning* from the pile of cosmetic reactions that should follow. This is the Chapter 6 + Chapter 8 division of labour in one screen of code.

Turn-based strategy: the other STR

Now the same world, turn-based. The structural change is that *nothing moves on its own* — the simulation advances only when the player ends their turn, and each unit has a budget of **action points (AP)** that movement and attacks spend. This is the *Civilization/X-COM* model, and it's a great showcase for Bevy **states** and a turn-cycle resource.

Rust – a turn cycle with per-side phases

```

#[derive(States, Debug, Clone, PartialEq, Eq, Hash, Default)]
enum TurnPhase { #[default] PlayerTurn, EnemyTurn }

#[derive(Resource)]
struct TurnCount(u32);

#[derive(Component)]
struct ActionPoints { current: u32, max: u32 } // e.g. 2 AP per unit per turn

// Ending the player's turn refills AP and hands control to the AI.
fn end_turn(
    keys: Res<ButtonInput<KeyCode>>,
    mut next: ResMut<NextState<TurnPhase>>,
    mut turn: ResMut<TurnCount>,
    mut units: Query<&mut ActionPoints>,
) {
    if keys.just_pressed(KeyCode::Enter) {
        for mut ap in &mut units { ap.current = ap.max; } // refill for next round
        turn.0 += 1;
        next.set(TurnPhase::EnemyTurn);
    }
}

```

Movement is no longer continuous: a click moves a unit **one tile** and costs **one AP**, and is refused when the unit is out of points. Tiles make distance discrete, so there are no fractions to integrate — just grid arithmetic.

```

#[derive(Component)]
struct GridPos { x: i32, y: i32 }

const TILE: f32 = 32.0;

// Order a selected unit to step to an adjacent tile, spending 1 AP.
fn move_on_grid(

```

```

    target: GridPos,
    mut q: Query<(&mut GridPos, &mut Transform, &mut ActionPoints), With<Selected>>,
) {
    for (mut pos, mut tf, mut ap) in &mut q {
        let dist = (target.x - pos.x).abs() + (target.y - pos.y).abs(); // Manhattan
        if dist == 1 && ap.current > 0 {
            pos.x = target.x;
            pos.y = target.y;
            ap.current -= 1; // movement costs a point
            tf.translation = Vec3::new(pos.x as f32 * TILE, pos.y as f32 * TILE, 0.0);
        }
    }
}

```

RTS VS STR IN ONE SENTENCE

Real-time strategy integrates movement *every tick* against a continuous clock (`FixedUpdate`, `delta_secs`); turn-based strategy advances *only on end-of-turn* and rations actions with a discrete resource (action points) on a tile grid. Same units, same economy, same orders — the entire difference is how time and distance are quantised. Pick the model that fits your game and reuse everything else.

FOR THE PYTHON DEVELOPER

The turn-based version is the one that feels most like ordinary Python game code: a clear "now it's your turn / now it's theirs" loop and integer grid maths, no frame-rate-dependent deltas. Bevy **states** give you that loop structure (`OnEnter(TurnPhase::EnemyTurn)` runs the AI once) without a hand-written state variable you have to remember to check everywhere.

What you built

A real-time half — click-to-world selection, right-click move orders, a deterministic `FixedUpdate` movement and economy simulation, and a production pipeline showing Messages and Observers together — and a turn-based half on the same units: a state-driven turn cycle, action points, and discrete tile movement. Scale the RTS selection to a drag-box and add pathfinding (Chapter 19's threaded A* fits here) for a C&C-style game; flesh out the AP system with attacks and an `OnEnter(TurnPhase::EnemyTurn)` AI for a Civ/X-COM-style one. Two genres, one shared foundation.

Workshop: Collectible Card Game

The inspiration: *Magic: The Gathering* (the 1993 original, digital descendants everywhere). A card game is almost pure *state and rules*: a turn structure, phases, a mana economy, a deck you shuffle and draw from. It's the workshop where Rust's enums and the **Command pattern** (Chapter 11) shine, and it barely touches sprites — so it's a great way to see game logic in isolation.

Modeling cards with enums

```
use bevy::prelude::*;

#[derive(Clone)]
enum Card {
    Creature { name: String, cost: u8, power: u8, toughness: u8 },
    Spell     { name: String, cost: u8, effect: SpellEffect },
}

#[derive(Clone, Copy)]
enum SpellEffect { Damage(u8), DrawCards(u8), GainMana(u8) }

impl Card {
    fn cost(&self) -> u8 {
        match self { Card::Creature { cost, .. } | Card::Spell { cost, .. } => *cost }
    }
}
```

FOR THE PYTHON DEVELOPER

In Python you'd reach for a class hierarchy or a dict with a `"type"` key and hope every branch handles every type. Rust's enum makes the set of card kinds *closed and checked*: add a new variant and the compiler lists every `match` you forgot to update.

Turn and phase state

A card game is a state machine of phases. Model phases as an enum and the game as a resource.

```
#[derive(Clone, Copy, PartialEq)]
enum Phase { Untap, Draw, Main, Combat, End }

#[derive(Resource)]
struct GameState {
    turn: u32,
    phase: Phase,
    mana: u8,
    hand: Vec<Card>,
    deck: Vec<Card>,
}

fn advance_phase(mut state: ResMut<GameState>) {
    state.phase = match state.phase {
        Phase::Untap => Phase::Draw,
        Phase::Draw  => Phase::Main,
        Phase::Main   => Phase::Combat,
    }
}
```

```

        Phase::Combat => Phase::End,
        Phase::End    => { state.turn += 1; Phase::Untap }
    };
}

```

Shuffling and drawing — with `rand` 0.9

Shuffling uses the `rand` crate. Note the **0.9 API**: `rand::rng()` (not the old `thread_rng()`), and `SliceRandom::shuffle`.

Rust – shuffle the deck, draw to hand

```

use rand::seq::SliceRandom;

fn shuffle_deck(deck: &mut Vec<Card>) {
    let mut rng = rand::rng(); // rand 0.9 entry point
    deck.shuffle(&mut rng);
}

fn draw(state: &mut GameState, n: usize) {
    for _ in 0..n {
        match state.deck.pop() {
            Some(card) => state.hand.push(card),
            None => { info!("deck empty — you lose on next draw"); break; }
        }
    }
}

```

DETERMINISTIC SHUFFLES FOR TESTING & REPLAYS

`rand::rng()` is seeded from the OS. For reproducible games (and the replays in Chapter 20) use a seeded generator like `StdRng::seed_from_u64(seed)` so the same seed always produces the same shuffle — invaluable when writing tests for your rules engine.

Playing a card via the Command pattern

Each play is a `GameCommand` (Chapter 11). Applying it mutates state; storing it gives you a log you can undo or serialise.

```

enum GameCommand {
    PlayCreature { index: usize },
    CastSpell     { index: usize },
}

fn apply(cmd: &GameCommand, state: &mut GameState) -> Result<(), String> {
    match cmd {
        GameCommand::PlayCreature { index } | GameCommand::CastSpell { index } => {
            let card = state.hand.get(*index).ok_or("no such card")?;
            if card.cost() > state.mana {
                return Err(format!("need {} mana, have {}", card.cost(), state.mana));
            }
            state.mana -= card.cost();
            let card = state.hand.remove(*index);
            resolve(card, state);
            Ok(())
        }
    }
}

```



```

    }
}

fn resolve(card: Card, state: &mut GameState) {
    match card {
        Card::Creature { name, .. } => info!("{name} enters the battlefield"),
        Card::Spell { effect, .. } => match effect {
            SpellEffect::Damage(d) => info!("deal {d} damage"),
            SpellEffect::DrawCards(n) => draw(state, n as usize),
            SpellEffect::GainMana(m) => state.mana += m,
        },
    },
}
}

```

THE RULES ENGINE WANTS TESTS, NOT A WINDOW

Because `apply` is a pure-ish function over `GameState`, you can unit-test the entire rules engine headlessly — "casting a 4-cost spell with 3 mana returns `Err`", "drawing from an empty deck is handled". Chapter 21 shows the pattern; a card game is the ideal candidate.

Wiring it to Bevy

```

pub struct CardGamePlugin;

impl Plugin for CardGamePlugin {
    fn build(&self, app: &mut App) {
        app
            .insert_resource(GameState {
                turn: 1, phase: Phase::Untap, mana: 3, hand: vec![], deck: vec![],
            })
            .add_systems(Update, (handle_input, advance_phase_on_space));
    }
}

```

Rendering the hand is then a matter of spawning a `Text` / `Node` per card (Chapter 9 UI) and reading `GameState.hand` — the logic above doesn't change.

What you built

A complete card-game core: enum-modeled cards, a phase state machine, a `rand` 0.9 shuffle and draw, and a Command-pattern play system that's serialisable and testable. The lesson travels: **model the rules as data and pure functions, render second**. Next, a genre that's all about *place* and interaction — the point-and-click adventure.

Workshop: Point-and-Click Adventure

The inspiration: *The Secret of Monkey Island* (1990) and the LucasArts SCUMM games — a verb you pick ("Look at", "Use", "Talk to"), an object you click, an inventory you combine, and branching dialogue. This workshop is about **interaction state**: a small machine that pairs a verb with a target and produces a response. Rust enums make the verb/target combinations exhaustive and impossible to leave half-handled.

The verb-target interaction model

```
use bevy::prelude::*;

#[derive(Clone, Copy, PartialEq)]
enum Verb { LookAt, Use, Pickup, TalkTo }

#[derive(Resource)]
struct Interaction { verb: Verb }    // the currently selected verb

#[derive(Component, Clone)]
struct Interactable {
    name: String,
    can_pickup: bool,
}
```

Selecting a verb is just writing the resource; clicking a hotspot reads it.

```
fn select_verb(keys: Res<ButtonInput<KeyCode>>, mut it: ResMut<Interaction>) {
    if keys.just_pressed(KeyCode::KeyL) { it.verb = Verb::LookAt; }
    if keys.just_pressed(KeyCode::KeyU) { it.verb = Verb::Use; }
    if keys.just_pressed(KeyCode::KeyP) { it.verb = Verb::Pickup; }
    if keys.just_pressed(KeyCode::KeyT) { it.verb = Verb::TalkTo; }
}
```

Resolving a click on a hotspot

```
#[derive(Resource, Default)]
struct Inventory { items: Vec<String> }

fn click_hotspot(
    buttons: Res<ButtonInput<MouseButton>>,
    windows: Query<&Window>,
    cameras: Query<(&Camera, &GlobalTransform)>,
    hotspots: Query<(Entity, &Transform, &Interactable)>,
    interaction: Res<Interaction>,
    mut inventory: ResMut<Inventory>,
    mut commands: Commands,
) -> Result {
    if !buttons.just_pressed(MouseButton::Left) { return Ok(()); }
    let window = windows.single()?;
    let (camera, cam_tf) = cameras.single()?;
    let Some(cursor) = window.cursor_position() else { return Ok(()); };
    let Ok(world) = camera.viewport_to_world_2d(cam_tf, cursor) else { return Ok(()); };
}
```

```

for (entity, tf, obj) in &hotspots {
    if tf.translation.truncate().distance(world) > 40.0 { continue; }

    // The exhaustive verb/target match – the heart of the genre.
    match interaction.verb {
        Verb::LookAt => info!("It's a {}.\"", obj.name),
        Verb::TalkTo => info!("You can't talk to a {}.\"", obj.name),
        Verb::Use     => info!("You use the {} ... nothing happens (yet).\"", obj.name),
        Verb::Pickup => {
            if obj.can_pickup {
                inventory.items.push(obj.name.clone());
                commands.entity(entity).despawn();
                info!("Picked up {}.\"", obj.name);
            } else {
                info!("You can't pick up the {}.\"", obj.name);
            }
        }
    }
    break; // resolve a single hotspot per click
}
Ok(())
}

```

FOR THE PYTHON DEVELOPER

A SCUMM-style game in Python often becomes a tangle of `if verb == "use" and obj == "key"` strings. The enum `match` here is the same idea with two safety nets: typos are compile errors, and adding a verb forces you to update every interaction. The `clone()` on `obj.name` is the deliberate kind — we're moving a name into the inventory and keeping the world tidy, not cloning every frame.

Combining inventory items

Adventure puzzles are usually "use item A on item B". Model the combination table as a function.

```

fn combine(a: &str, b: &str) -> Option<String> {
    // order-independent matching
    let pair = {
        let mut p = [a, b];
        p.sort();
        (p[0], p[1])
    };
    match pair {
        ("grog", "mug")    => Some("mug of grog".into()),
        ("key", "rusty")   => Some("oiled key".into()),
        _                  => None, // these two don't combine
    }
}

```

DATA-DRIVE THE PUZZLES

For a real game, load the combination and response tables from a `RON` or JSON file (Chapter 20) instead of hard-coding `match` arms. Designers can then add puzzles without touching Rust, and you can hot-reload them.

Branching dialogue

Dialogue is a graph: each node has lines and a set of player choices that jump to other nodes. A `HashMap<NodeId, DialogueNode>` plus a "current node" resource is enough.

```
use std::collections::HashMap;

struct Choice { text: String, goto: u32 }
struct DialogueNode { speaker: String, line: String, choices: Vec<Choice> }

#[derive(Resource)]
struct Dialogue { nodes: HashMap<u32, DialogueNode>, current: Option<u32> }

fn choose(dialogue: &mut Dialogue, choice_index: usize) {
    if let Some(id) = dialogue.current {
        if let Some(node) = dialogue.nodes.get(&id) {
            if let Some(choice) = node.choices.get(choice_index) {
                dialogue.current = Some(choice.goto); // walk the graph
            }
        }
    }
}
```

THIS IS THE VISUAL-NOVEL ENGINE TOO

A dialogue graph is exactly what the next workshop scales up. If you build it cleanly here, the visual novel is mostly presentation on top of the same structure.

What you built

A verb/target interaction machine with exhaustive handling, a pickup-and-despawn inventory, an order-independent item-combination table, and a graph-based dialogue system. The recurring win: **enums turn "did I handle every case?" from a runtime worry into a compile-time guarantee.** Next we lean all the way into simulation with a city builder.

Workshop: City Builder

The inspiration: *SimCity 2000* (1993) — a tile grid you zone into residential, commercial, and industrial; a budget; growth that emerges from simple per-tile rules ticked on a clock. This workshop is the payoff for Chapter 7's `FixedUpdate` and Chapter 10's change detection: a city is tens of thousands of tiles, and the whole craft is *touching as few of them as possible each tick*.

The tile grid

A grid maps cleanly to either a 2D `Vec` resource or one entity per tile. For a sim with per-tile components and change detection, entities win.

```
use bevy::prelude::*;

#[derive(Clone, Copy, PartialEq)]
enum ZoneKind { Empty, Residential, Commercial, Industrial }

#[derive(Component)] struct Tile { x: i32, y: i32 }
#[derive(Component)] struct Zone(ZoneKind);
#[derive(Component)] struct Population(u32);

#[derive(Resource)] struct Treasury { funds: i64 }
#[derive(Resource)] struct SimTimer(Timer); // the growth clock
```

```
fn spawn_grid(mut commands: Commands) {
    commands.spawn(Camera2d);
    for y in 0..64 {
        for x in 0..64 {
            commands.spawn((
                Tile { x, y },
                Zone(ZoneKind::Empty),
                Population(0),
                Transform::from_xyz(x as f32 * 16.0, y as f32 * 16.0, 0.0),
                Sprite { color: Color::srgb(0.15, 0.18, 0.15),
                        custom_size: Some(Vec2::splat(15.0)), ..default() },
            ));
        }
    }
}
```

Zoning: paint tiles, recolor only what changed

Painting is a click that mutates one tile's `Zone`. Recoloring is a *separate* system filtered by `Changed<Zone>` so the renderer only touches edited tiles — not all 4,096 every frame.

```
fn paint_zone(
    buttons: Res<ButtonInput<MouseButton>>,
    keys: Res<ButtonInput<KeyCode>>,
    windows: Query<&Window>,
    cameras: Query<(&Camera, &GlobalTransform)>,
    mut tiles: Query<(&Transform, &mut Zone)>,
) -> Result {
    if !buttons.pressed(MouseButton::Left) { return Ok(()); }
```

```

let window = windows.single()?;
let (camera, cam_tf) = cameras.single()?;
let Some(cursor) = window.cursor_position() else { return Ok(()); };
let Ok(world) = camera.viewport_to_world_2d(cam_tf, cursor) else { return Ok(()); };

let kind = if keys.pressed(KeyCode::KeyC) { ZoneKind::Commercial }
            else if keys.pressed(KeyCode::KeyI) { ZoneKind::Industrial }
            else { ZoneKind::Residential };

for (tf, mut zone) in &mut tiles {
    if tf.translation.truncate().distance(world) < 10.0 && zone.0 != kind {
        zone.0 = kind; // mutating flags this tile as Changed
    }
}
Ok(())
}

fn recolor_changed(mut q: Query<(&Zone, &mut Sprite), Changed<Zone>>()) {
    for (zone, mut sprite) in &mut q {
        sprite.color = match zone.0 {
            ZoneKind::Empty      => Color::srgb(0.15, 0.18, 0.15),
            ZoneKind::Residential => Color::srgb(0.2, 0.7, 0.3),
            ZoneKind::Commercial => Color::srgb(0.3, 0.5, 0.9),
            ZoneKind::Industrial => Color::srgb(0.9, 0.8, 0.3),
        };
    }
}

```

THE `!= KIND` GUARD MATTERS

Without `&& zone.0 != kind`, dragging over an already-correct tile would still take `&mut` and re-flag it `Changed`, defeating the optimisation and re-running the recolor. Cheap equality check, big saving. (Recall the Chapter 10 caution: `&mut` access alone marks changed.)

Growth: a deterministic FixedUpdate tick

Population growth is the simulation. It runs on a clock, reads neighbours, and is fully reproducible — exactly what `FixedUpdate` is for. Here, residential tiles grow toward a cap.

```

fn grow_city(
    time: Res<Time>,
    mut timer: ResMut<SimTimer>,
    mut q: Query<(&Zone, &mut Population)>,
) {
    if !timer.0.tick(time.delta()).just_finished() { return; }
    for (zone, mut pop) in &mut q {
        if zone.0 == ZoneKind::Residential && pop.0 < 100 {
            pop.0 += 5; // simple growth rule; real sims read neighbours
        }
    }
}

```

READING NEIGHBOURS WITHOUT BORROW PAIN

To make growth depend on adjacency (residential grows faster next to commercial), use the Chapter 5 read-then-write split: in a read pass, build a `HashMap<(i32,i32), ZoneKind>` of the grid; in a write pass, look up each tile's neighbours in that snapshot and update. Snapshotting sidesteps the "can't read the grid while mutating it" borrow conflict and keeps the tick deterministic.

Budget

```
fn collect_taxes(
    time: Res<Time>,
    mut timer: ResMut<SimTimer>,
    mut treasury: ResMut<Treasury>,
    q: Query<&Population>,
) {
    if !timer.0.just_finished() { return; } // same tick as growth
    let total: u32 = q.iter().map(|p| p.0).sum();
    treasury.funds += (total as i64) / 10; // tax revenue
    treasury.funds -= 50; // upkeep
}
```

A note on data layout at city scale

A 256×256 city is 65,536 tiles. Keep the per-tile components tiny (`Zone` is one byte, `Population` four). Anything heavy and rarely-touched — a history log, a detailed building model — goes in its own component so it lives in a separate array and never pollutes the cache during the hot growth and render passes (Chapter 10's split-hot-from-cold rule).

What you built

A zonable tile grid, painting that only recolors changed tiles, a deterministic `FixedUpdate` growth tick, and a budget — the skeleton of a city sim that stays fast at scale. The big-picture skill is *simulation discipline*: fixed timestep, change detection, and small hot components. Next, the opposite pace — a story-driven visual novel.

Workshop: Visual Novel

The inspiration: *Tokimeki Memorial* (1994) and the dating-sim/visual-novel lineage — scripted scenes, character portraits, branching choices, and hidden affection variables that steer the ending. Mechanically this is the Chapter 15 dialogue graph grown into a full engine, plus **flags and variables** that persist across the story. It's also a showcase for a small but delightful effect: the typewriter text reveal.

The script as data

A novel is a list (or map) of nodes. Each node shows a line and either advances automatically or offers choices that jump elsewhere and tweak variables.

```
use bevy::prelude::*;
use std::collections::HashMap;

struct Choice { text: String, goto: usize, affection: i32 }

struct ScriptNode {
    speaker: String,
    portrait: String,          // asset path
    text: String,
    choices: Vec<Choice>,      // empty = press to continue to `goto`
    goto: usize,               // used when there are no choices
}

#[derive(Resource)]
struct Story {
    nodes: Vec<ScriptNode>,
    current: usize,
    vars: HashMap<String, i32>, // flags & counters: "shiori_affection", "met_yuko", ...
}
```

FOR THE PYTHON DEVELOPER

This is the part that feels most like Python: a `HashMap<String, i32>` is your `dict` of game flags, and you read/write it exactly as you would in a Ren'Py script. The difference is that the *structure* around it — nodes, choices — is statically typed, so a malformed branch is caught when you build, not when a player hits it three hours in.

Advancing and branching

```
fn advance(story: &mut Story, choice_index: Option<usize>) {
    let node = &story.nodes[story.current];
    let next = match choice_index {
        Some(i) => {
            let choice = &node.choices[i];
            // apply the side effect of the choice
            *story.vars.entry("affection".into()).or_insert(0) += choice.affection;
            choice.goto
        }
        None => node.goto, // no choice: linear continue
    };
};
```



```
    story.current = next;
}
```

The `entry(...).or_insert(0)` idiom (Chapter 2) is how you safely bump a flag that may not exist yet — the visual-novel bread and butter.

Conditional branching on variables

Endings and gated scenes depend on accumulated variables. Keep the condition logic in one place.

```
fn ending_for(story: &Story) -> usize {
    let aff = story.vars.get("affection").copied().unwrap_or(0);
    match aff {
        a if a >= 50 => 900,    // good ending node
        a if a >= 20 => 901,    // neutral ending node
        _              => 902,    // bad ending node
    }
}
```

The typewriter effect

Revealing text one character at a time is the signature visual-novel flourish. Track how many characters to show and grow it on a timer; clicking skips to the full line.

Rust — character-by-character reveal

```
#[derive(Component)]
struct Typewriter { full: String, shown: usize, timer: Timer }

fn typewriter_tick(
    time: Res<Time>,
    buttons: Res<ButtonInput<MouseButton>>,
    mut q: Query<(&mut Typewriter, &mut Text)>,
) {
    for (mut tw, mut text) in &mut q {
        if buttons.just_pressed(MouseButton::Left) {
            tw.shown = tw.full.chars().count();    // click = reveal all
        } else if tw.timer.tick(time.delta()).just_finished()
            && tw.shown < tw.full.chars().count() {
            tw.shown += 1;    // one more glyph
        }
        // Rebuild the visible slice by char count (UTF-8 safe — see warning).
        text.0 = tw.full.chars().take(tw.shown).collect();
    }
}
```

COUNT CHARACTERS, NOT BYTES

Never slice a Rust `String` with `&s[0..n]` to reveal text: indices are *bytes*, and Japanese, accented, or emoji characters are multiple bytes — a byte slice can land mid-character and **panic**. Use `.chars().take(n).collect()` as above. For a dating sim that may ship in Japanese, this isn't optional.

Wiring portraits and the box

Presentation is Chapter 9 in miniature: a full-screen background sprite, a portrait sprite swap per node, and a `Text` node for the dialogue box. The engine logic above never changes when you restyle the UI.

```
fn show_node(story: Res<Story>, assets: Res<AssetServer>, mut portraits: Query<&mut Sprite,
With<Portrait>>) {
    if !story.is_changed() { return; } // only refresh when the node advanced
    let node = &story.nodes[story.current];
    if let Ok(mut sprite) = portraits.single_mut() {
        sprite.image = assets.load(&node.portrait);
    }
}
```

IS_CHANGED() ON RESOURCES

`Res<T>` exposes `is_changed()`, the resource-level cousin of the `Changed<T>` query filter. Gating presentation on it means you only reload a portrait when the story actually moves to a new node, not sixty times a second.

What you built

A data-driven script engine with branching choices, a `HashMap` of persistent flags and affection variables, variable-gated endings, and a UTF-8-safe typewriter effect. The headline lesson for text-heavy games: **respect Unicode** — work in `chars`, never raw byte indices. Last workshop, and the most technically ambitious: a first-person shooter in 3D.

Workshop: First-Person Shooter

The inspiration: *Wolfenstein 3D* (1992) and *Doom* (1993) — first-person movement, mouse look, and hitscan shooting down a corridor of enemies. This is the only 3D workshop, so it adds the third dimension to everything you've learned: a `Camera3d`, meshes and materials, yaw/pitch from the mouse, WASD relative to where you're facing, and a ray-vs-sphere hitscan.

CONSERVATIVE 3D

3D APIs move faster than 2D between Bevy versions. This workshop deliberately uses only the stable core — `Camera3d`, `Mesh3d`, `MeshMaterial3d`, `Transform` — and does its own raycast maths rather than depending on a physics or picking crate, so the code keeps compiling as the ecosystem churns.

Scene setup: camera, light, floor

```
use bevy::prelude::*;

fn setup(
    mut commands: Commands,
    mut meshes: ResMut<Assets<Mesh>>,
    mut materials: ResMut<Assets<StandardMaterial>>,
) {
    // First-person camera = a Camera3d we move and rotate each frame.
    commands.spawn((
        Camera3d::default(),
        Transform::from_xyz(0.0, 1.7, 0.0).looking_to(Vec3::NEG_Z, Vec3::Y),
        Player { yaw: 0.0, pitch: 0.0 },
    ));

    // A directional light so the world isn't pitch black.
    commands.spawn((
        DirectionalLight { illuminance: 10_000.0, ..default() },
        Transform::from_xyz(4.0, 8.0, 4.0).looking_at(Vec3::ZERO, Vec3::Y),
    ));

    // Floor plane.
    commands.spawn((
        Mesh3d(meshes.add(Plane3d::default().mesh().size(50.0, 50.0))),
        MeshMaterial3d(materials.add(Color::srgb(0.3, 0.3, 0.35))),
    ));
}

#[derive(Component)]
struct Player { yaw: f32, pitch: f32 }
```

Mouse look

Accumulate yaw (left/right) and pitch (up/down) from raw mouse motion **messages** (Chapter 9), clamp the pitch so you can't somersault, and rebuild the camera rotation.

```
use bevy::input::mouse::MouseEvent;

fn mouse_look(
```

```

    mut motion: MessageReader<MouseMotion>,
    mut q: Query<(&mut Transform, &mut Player)>,
) -> Result {
    let (mut tf, mut player) = q.single_mut()?;
    let sensitivity = 0.002;

    for ev in motion.read() {
        player.yaw -= ev.delta.x * sensitivity;
        player.pitch -= ev.delta.y * sensitivity;
    }
    // Clamp pitch to just under straight up/down.
    player.pitch = player.pitch.clamp(-1.54, 1.54);

    tf.rotation = Quat::from_axis_angle(Vec3::Y, player.yaw)
        * Quat::from_axis_angle(Vec3::X, player.pitch);
    Ok(())
}

```

GRAB THE CURSOR

For real mouse look you want the OS cursor locked and hidden so it can't leave the window. Set the window's `cursor_options.grab_mode` to locked and `visible` to false (the exact field path shifts between versions — check Appendix B / the migration guide for 0.18).

Movement relative to facing

WASD must move *where you're looking*, not along fixed world axes. Derive forward/right from the camera transform. To get a `Vec3` from the `Dir3` that `forward()` returns, convert it — this is the version-stable way.

Rust – facing-relative movement, flattened to the ground plane

```

fn move_player(
    keys: Res<ButtonInput<KeyCode>>,
    time: Res<Time>,
    mut q: Query<(&mut Transform, With<Player>>),
) -> Result {
    let mut tf = q.single_mut()?;
    let speed = 5.0 * time.delta_secs();

    // Dir3 -> Vec3 via From; flatten Y so you don't fly when looking up.
    let mut forward = Vec3::from(tf.forward());
    let mut right = Vec3::from(tf.right());
    forward.y = 0.0; right.y = 0.0;
    forward = forward.normalize_or_zero();
    right = right.normalize_or_zero();

    let mut delta = Vec3::ZERO;
    if keys.pressed(KeyCode::KeyW) { delta += forward; }
    if keys.pressed(KeyCode::KeyS) { delta -= forward; }
    if keys.pressed(KeyCode::KeyD) { delta += right; }
    if keys.pressed(KeyCode::KeyA) { delta -= right; }

    tf.translation += delta.normalize_or_zero() * speed;
    Ok(())
}

```

NORMALIZE AFTER SUMMING, AND GUARD THE ZERO VECTOR

Add the direction vectors first, then normalize the result — otherwise diagonal movement is faster than cardinal (the classic "strafe-run" bug). `normalize_or_zero()` is essential: plain `normalize()` on a zero vector yields `NaN`, which silently corrupts the transform and is miserable to debug.

Hitscan shooting: ray vs sphere

A hitscan weapon fires an instant ray from the camera. We test it against enemy bounding spheres and keep the nearest hit. The maths is a ray-sphere intersection — small, stable, dependency-free.

```
#[derive(Component)]
struct Enemy { radius: f32 }

fn shoot(
    buttons: Res<ButtonInput<MouseButton>>,
    cam: Query<&Transform, With<Player>>,
    enemies: Query<(Entity, &Transform, &Enemy)>,
    mut commands: Commands,
) -> Result {
    if !buttons.just_pressed(MouseButton::Left) { return Ok(()); }
    let cam_tf = cam.single()?;
    let origin = cam_tf.translation;
    let dir = Vec3::from(cam_tf.forward()); // unit ray direction

    let mut best: Option<(Entity, f32)> = None;
    for (e, tf, enemy) in &enemies {
        if let Some(t) = ray_sphere(origin, dir, tf.translation, enemy.radius) {
            if best.map_or(true, |(_, bt)| t < bt) {
                best = Some((e, t)); // nearest enemy along the ray
            }
        }
    }
    if let Some((hit, _)) = best {
        commands.entity(hit).despawn();
        info!("hit!");
    }
    Ok(())
}

/// Returns the distance along the ray to the first intersection, if any.
fn ray_sphere(origin: Vec3, dir: Vec3, center: Vec3, radius: f32) -> Option<f32> {
    let oc = origin - center;
    let b = oc.dot(dir);
    let c = oc.length_squared() - radius * radius;
    let disc = b * b - c; // discriminant
    if disc < 0.0 { return None; } // ray misses the sphere
    let t = -b - disc.sqrt(); // nearest root
    if t >= 0.0 { Some(t) } else { None } // ignore hits behind the camera
}
```

FOR THE PYTHON DEVELOPER

This is the moment Rust's speed pays a visible dividend: the per-frame ray test over every enemy, plus the movement and look maths, all run in native code with no allocation. The same loop in CPython would be your frame-budget bottleneck; here it's noise. For a *large* arena, the Chapter 10 spatial-grid idea narrows the candidate set before the ray test.

What you built

A first-person camera with clamped mouse look, facing-relative WASD movement with the diagonal and NaN bugs handled, and a dependency-free hitscan via ray-sphere intersection — the core of a Doom-style shooter, in version-stable 3D. That completes the genre tour. Part VII zooms back out to the standard library that underpins all of it.

PART VII · THE STANDARD LIBRARY FIELD GUIDE

The Standard Library for Games

Rust's standard library is small but sharp. This chapter is a tour of the parts you'll reach for constantly in a game, adapted from the *Rust Standard Library* documentation into game-shaped recipes: the collections beyond `Vec`, string parsing for consoles and dialogue, file I/O for saves, error handling that scales, threads for heavy work, and where the maths lives.

Collections beyond Vec

You met `Vec`, `HashMap`, `HashSet`, and `BTreeMap` in Chapter 2. Here are the game-specific moves.

VecDeque — a queue for orders and events

A `VecDeque` is a double-ended queue: push to one end, pop from the other in $O(1)$. Perfect for unit order queues, a turn order, or an input buffer.

```
use std::collections::VecDeque;

#[derive(Component)]
struct OrderQueue(VecDeque<Order>);

fn process_orders(mut q: Query<&mut OrderQueue>) {
    for mut oq in &mut q {
        if let Some(order) = oq.0.pop_front() { // oldest order first
            // execute one order per tick
        }
    }
}

// enqueue with oq.0.push_back(order);
```

BTreeMap — a sorted leaderboard

A `BTreeMap` keeps keys in sorted order, so iterating gives you a ranked list for free.

Rust – a high-score table (note the rank fix)

```
use std::collections::BTreeMap;

fn print_leaderboard(scores: &BTreeMap<u32, String>) {
    // Iterate high-to-low and number the rows with enumerate().
    for (rank, (score, name)) in scores.iter().rev().enumerate() {
        println!("#{:}: {} - {} pts", rank + 1, name, score);
    }
}
```

THE BUG THIS FIXES

An earlier draft printed the rank as `scores.len()` inside the loop — so every row claimed to be the same rank (the total number of entries). The fix is `.enumerate()`, which yields a running index; `rank + 1` makes it human-friendly (1-based). A tiny change, but it's the difference between a working leaderboard and nonsense. Always derive the rank from iteration, never from the collection's length.

HashSet — fast membership and de-duplication

```
use std::collections::HashSet;

// Which tiles has the player discovered? O(1) "have I seen this?".
#[derive(Resource, Default)]
struct Explored(HashSet<(i32, i32)>);

fn reveal(mut explored: ResMut<Explored>, tile: (i32, i32)) {
    let is_new = explored.0.insert(tile); // returns false if already present
    if is_new { /* award exploration XP, update fog */ }
}
```

Strings and parsing

Game consoles, chat commands, and config lines all need parsing. Rust's `str` methods plus `parse::<T>()` cover most of it without a library.

Rust – a tiny debug-console command parser

```
fn run_command(input: &str) -> Result<String, String> {
    let mut parts = input.split_whitespace();
    let cmd = parts.next().ok_or("empty command")?;
    match cmd {
        "give" => {
            let item = parts.next().ok_or("usage: give <item> <count>")?;
            let count: u32 = parts.next().unwrap_or("1")
                .parse().map_err(|_| "count must be a number")?;
            Ok(format!("gave {count}x {item}"))
        }
        "tp" => {
            let x: f32 = parts.next().ok_or("usage: tp <x> <y>")?.parse().map_err(|_| "bad
x")?;
            let y: f32 = parts.next().ok_or("usage: tp <x> <y>")?.parse().map_err(|_| "bad
y")?;
            Ok(format!("teleported to ({x}, {y})"))
        }
        other => Err(format!("unknown command: {other}")),
    }
}
```

FOR THE PYTHON DEVELOPER

`input.split_whitespace()` is `str.split()`, `parts.next()` is iterating the result, and `.parse::<u32>()` is `int(...)` — except `parse` returns a `Result` you must handle instead of raising `ValueError`. The `?` operator threads all those failures out to one place, the way a single `try/except` would wrap the whole function.

File I/O: saving and loading

Reading and writing files is in `std::fs`. Pair it with `serde` + `serde_json` to persist game state as JSON.

```
use serde::{Serialize, Deserialize};
```



```
#[derive(Serialize, Deserialize)]
struct SaveGame { level: u32, gold: u32, inventory: Vec<String> }

fn save(state: &SaveGame, path: &str) -> std::io::Result<()> {
    let json = serde_json::to_string_pretty(state).expect("serialize");
    std::fs::write(path, json) // creates/overwrites the file
}

fn load(path: &str) -> std::io::Result<SaveGame> {
    let text = std::fs::read_to_string(path)?; // ? propagates a missing-file error
    let state = serde_json::from_str(&text)
        .map_err(|e| std::io::Error::new(std::io::ErrorKind::InvalidData, e))?;
    Ok(state)
}
```

Chapter 20 builds on this with versioned saves and migration so old save files keep working.

Error handling that scales

`unwrap()` is fine in prototypes; real games want typed errors. A common pattern is one error enum per subsystem, with `?` doing the propagation.

```
#[derive(Debug)]
enum AssetError { NotFound(String), BadFormat(String) }

impl std::fmt::Display for AssetError {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        match self {
            AssetError::NotFound(p) => write!(f, "asset not found: {p}"),
            AssetError::BadFormat(p) => write!(f, "bad format: {p}"),
        }
    }
}

impl std::error::Error for AssetError {}
```

CRATES THAT REMOVE THE BOILERPLATE

For application code, `thiserror` derives the `Display` / `Error` impls above from attributes, and `anyhow` gives you a catch-all error type for the top level. They're the de facto standard; reach for them once your hand-written error enums get tedious.

Concurrency: threads for the heavy stuff

Game logic stays single-threaded (the ECS schedules parallelism for you), but *offline* work — procedural generation, pathfinding, chunk meshing — can run on a worker thread and report back through a channel, so it never stalls the frame.

Rust — generate a map on a worker thread, receive it when ready

```
use std::sync::mpsc;
use std::thread;

#[derive(Resource)]
struct MapGen { rx: mpsc::Receiver<Vec<u8>> }

fn start_generation(mut commands: Commands) {
```

```

let (tx, rx) = mpsc::channel();
thread::spawn(move || {
    let map = expensive_world_gen(); // runs off the main thread
    let _ = tx.send(map);           // hand the result back
});
commands.insert_resource(MapGen { rx });
}

fn poll_generation(gen: Res<MapGen>, mut commands: Commands) {
    if let Ok(map) = gen.rx.try_recv() { // non-blocking: returns Err if not ready
        // build the world from `map`
        commands.remove_resource::();
    }
}

```

SHARED MUTABLE STATE ACROSS THREADS NEEDS `ARC<MUTEX<T>>`

A channel *moves* data between threads, which is clean and lock-free. If multiple threads must touch the *same* data, wrap it in `Arc<Mutex<T>>` (Chapter 4) — but prefer message passing where you can. Bevy also offers async task pools (`AsyncComputeTaskPool`) that fit this pattern more idiomatically than raw `std::thread` for engine work.

Math and randomness

You don't need a math library — Bevy re-exports **glam** (`Vec2`, `Vec3`, `Quat`, `Mat4`) and that's what every example here uses. For randomness, the ecosystem standard is the `rand` crate, version **0.9**, whose entry point is `rand::rng()`.

```

use rand::Rng;

fn random_spawn_point() -> Vec2 {
    let mut rng = rand::rng(); // rand 0.9
    Vec2::new(rng.random_range(-400.0..400.0), // .random_range in 0.9
              rng.random_range(-300.0..300.0))
}

```

`RAND` 0.9 RENAMES

If you follow older tutorials you'll see `thread_rng()` and `gen_range()`. In 0.9 those are `rng()` and `random_range()`. The Bevy 0.17 upgrade pulled in this newer `rand`, so the new names are what compiles against a 0.18 project.

What we covered

The collections that matter for games (`VecDeque`, `BTreeMap`, `HashSet`) with the leaderboard rank bug fixed, string parsing for consoles, file I/O with `serde`, scalable error handling, threads-plus-channels for offline work, and where math and randomness live. These are the std foundations the cookbook recipes in Part VIII assemble into finished features.

PART VIII · THE GAME DEVELOPER'S COOKBOOK

Practical Recipes

Where the standard-library chapter taught ingredients, this one cooks finished dishes. Each recipe is a small, complete solution to a problem every real game hits, adapted from the spirit of the *Rust Cookbook* into Bevy 0.18: versioned save files, human-editable config, logging, timing a system, deterministic replays, and loading data tables. Steal them directly.

Recipe: versioned save files with migration

Saves outlive your code. If you add a field in v2, last week's v1 save must still load. The trick: store a version number and migrate forward.

Rust – tagged enum versions, migrated to the latest on load

```
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize)]
#[serde(tag = "version")] // each variant tags itself with its version
enum SaveFile {
    V1 { level: u32, gold: u32 },
    V2 { level: u32, gold: u32, difficulty: u8 }, // added a field
}

// The shape the game actually uses today.
struct GameSave { level: u32, gold: u32, difficulty: u8 }

fn load_and_migrate(text: &str) -> Result<GameSave, serde_json::Error> {
    let file: SaveFile = serde_json::from_str(text)?;
    Ok(match file {
        // Old saves get a sensible default for the new field.
        SaveFile::V1 { level, gold } => GameSave { level, gold, difficulty: 1 },
        SaveFile::V2 { level, gold, difficulty } => GameSave { level, gold, difficulty },
    })
}
```

ALWAYS WRITE THE NEWEST VERSION

Load *any* version, but always *save* the latest (`SaveFile::V2`). Over time every save migrates up. Keep old variants around as long as players might still have those files — the enum makes "have I handled every old version?" a compile-time check.

Recipe: human-editable config with RON or TOML

JSON is fine for saves; for files *humans* edit (settings, tuning, level data) prefer **RON** (Rusty Object Notation — it looks like Rust) or **TOML**. Both deserialize through `serde` exactly like JSON.

```
use serde::Deserialize;

#[derive(Deserialize)]
struct Settings {
    master_volume: f32,
    fullscreen: bool,
    key_bindings: Vec<(String, String)>,
}
```

```

}

fn load_settings(path: &str) -> Settings {
    let text = std::fs::read_to_string(path).unwrap_or_default();
    ron::from_str(&text).unwrap_or(Settings {           // fall back to defaults
        master_volume: 0.8, fullscreen: false, key_bindings: vec![],
    })
}

```

RON IS NOW A DIRECT DEPENDENCY

Bevy 0.18 stopped re-exporting `ron` from `bevy_asset` / `bevy_scene`. If you use it, add `ron = "0.8"` to your own `Cargo.toml` rather than reaching through Bevy — a small but real migration gotcha from the 0.17→0.18 change.

Recipe: logging instead of println!

Bevy ships a logging system. Use its macros, not `println!`: they carry severity levels, can be filtered by environment variable, and don't slow you down when disabled.

```

fn combat_system(/* ... */) {
    trace!("per-frame spam, usually hidden");
    debug!("enemy count: {}", 42);
    info!("wave 3 started");
    warn!("texture missing, using placeholder");
    error!("save failed: disk full");
}

```

```

# Control verbosity at runtime without recompiling:
RUST_LOG=warn cargo run           # only warnings and errors
RUST_LOG=my_game=debug cargo run  # debug for your crate, quiet for deps

```

FOR THE PYTHON DEVELOPER

This is exactly Python's `logging` module: levels, per-module filtering, and a runtime switch. `info!` is `logging.info`, and `RUST_LOG` is your `logging.basicConfig(level=...)` — except set from the shell so you can change it per run.

Recipe: timing a system (lightweight profiling)

Before you optimise (Chapter 10), measure. The RAI timer from Chapter 11 makes this a one-liner; here's the standalone version.

```

fn pathfinding_system(/* ... */) {
    let start = std::time::Instant::now();

    // ... the work you suspect is slow ...

    let elapsed = start.elapsed();
    if elapsed.as_millis() > 4 {                // flag anything eating the frame budget
        warn!("pathfinding took {:?}", elapsed);
    }
}

```

DON'T GUESS, MEASURE — THEN REACH FOR THE REAL TOOLS

A 60 fps frame is ~16 ms; if one system eats 4 ms it deserves attention. For serious work, Bevy integrates with the **Tracy** profiler (enable the `trace_tracy` feature) which shows a flame graph of every system per frame. Start with the cheap `Instant` check above to find the suspect, then bring in Tracy to understand it.

Recipe: deterministic RNG for replays

A replay (or a network lockstep sim, or a reproducible test) needs the *same* random numbers every run. Seed a generator and store the seed.

```
use rand::{SeedableRng, Rng};
use rand::rngs::StdRng;

#[derive(Resource)]
struct GameRng(StdRng);

fn init_rng(mut commands: Commands) {
    let seed = 0xC0FFEE_u64; // store this in the save/replay
    commands.insert_resource(GameRng(StdRng::seed_from_u64(seed)));
}

fn spawn_loot(mut rng: ResMut<GameRng>) {
    let roll: f32 = rng.0.random(); // same seed → same rolls, every time
    if roll > 0.95 { /* rare drop */ }
}
```

ONE SEED, ONE GENERATOR, ONE ORDER

Determinism breaks if you sometimes use the seeded RNG and sometimes `rand::rng()`, or if systems consume random numbers in a different order between runs. Route *all* gameplay randomness through the one seeded `Resource`, and pin system order (Chapter 7) so the draw sequence is stable.

Recipe: loading a data table

Designers tune games by editing tables, not Rust. Load a list of item/enemy/weapon definitions from a file at startup into a resource keyed for lookup.

```
use serde::Deserialize;
use std::collections::HashMap;

#[derive(Deserialize, Clone)]
struct WeaponDef { name: String, damage: f32, fire_rate: f32 }

#[derive(Resource, Default)]
struct WeaponTable(HashMap<String, WeaponDef>);

fn load_weapons(mut commands: Commands) {
    let text = std::fs::read_to_string("assets/data/weapons.ron").unwrap_or_default();
    let list: Vec<WeaponDef> = ron::from_str(&text).unwrap_or_default();
    let table = list.into_iter().map(|w| (w.name.clone(), w)).collect();
    commands.insert_resource(WeaponTable(table));
}
```

HOT-RELOAD FOR A TIGHT LOOP

Because the table lives in a file, you can re-read it on a key press (or watch the file) and re-tune weapons *while the game runs*. Bevy's asset system can hot-reload custom assets too; for quick iteration, even a manual "press F5 to reload data" system pays for itself many times over during balancing.

What we covered

Six drop-in recipes: forward-migrating versioned saves, RON/TOML config with graceful defaults, levelled logging with runtime filtering, lightweight system timing, seeded RNG for replays and tests, and data-table loading for designer-friendly tuning. These turn the language and engine features into the *operational* machinery a shippable game needs. The final chapter makes sure it all keeps working: testing and debugging.

PART IX · SHIPPING QUALITY

Testing & Debugging Bevy

Rust's tooling makes testing a first-class, built-in activity — no framework to install, `cargo test` just works. For a game, tests are what let you refactor the rules engine or the economy without fear, and they're especially worth it for the logic-heavy genres (cards, city builder, RTS). This chapter shows how to test pure functions, how to test *systems* by stepping a headless `App`, and the Bevy-specific traps that bite at runtime.

Testing pure functions

The easiest and highest-value tests target plain functions — damage formulas, the card-cost check, the leaderboard sort, the command parser. Put them in a `#[cfg(test)]` module in the same file.

Rust — unit tests live next to the code

```
fn apply_damage hp: f32, dmg: f32, armor: f32 -> f32 {
    (hp - (dmg - armor).max(0.0)).max(0.0)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn armor_reduces_damage() {
        assert_eq!(apply_damage(100.0, 30.0, 10.0), 80.0);
    }

    #[test]
    fn damage_never_pushes_hp_below_zero() {
        assert_eq!(apply_damage(5.0, 999.0, 0.0), 0.0);
    }

    #[test]
    fn armor_cannot_heal() {
        // armor higher than damage shouldn't add HP
        assert_eq!(apply_damage(50.0, 5.0, 100.0), 50.0);
    }
}
```

`cargo test` # runs every `#[test]` in the crate

FOR THE PYTHON DEVELOPER

`#[cfg(test)]` is `if __name__ == '__main__'` for tests, except the compiler strips the module entirely from release builds. `#[test]` is `pytest`'s test discovery, `assert_eq!` is `assert a == b`, and `cargo test` is `pytest`. No `conftest.py`, no install — it's in the language.

Testing systems by stepping an App

The powerful technique: build a minimal headless `App`, insert your system and some entities, call `app.update()` to run one frame, then assert on the resulting world state. This tests real ECS behaviour — queries, messages, commands — without opening a window.

Rust — drive a system one tick and inspect the world

```
use bevy::prelude::*;

#[derive(Component)] struct Health(f32);
#[derive(Component)] struct Poisoned;

fn poison_tick(mut q: Query<&mut Health, With<Poisoned>>) {
    for mut hp in &mut q { hp.0 -= 5.0; }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn poison_drains_five_hp_per_tick() {
        let mut app = App::new(); // no DefaultPlugins → headless & fast
        app.add_systems(Update, poison_tick);

        let entity = app.world_mut().spawn((Health(20.0), Poisoned)).id();

        app.update(); // run exactly one frame

        let hp = app.world().get::

```

STEP SEVERAL FRAMES TO TEST TIMERS AND STATE

Call `app.update()` in a loop to advance multiple frames — handy for testing that a wave timer fires after N ticks, or that a state transition lands. You can also insert resources (`app.insert_resource(...)`) to set up economy or RNG state, and read them back after the update to assert the outcome.

Testing messages and state transitions

The same harness verifies decoupled communication: write a message, run a frame, and check the reader's effect — or set a `State` and confirm the right systems ran.


```
#[test]
fn build_request_spends_credits() {
    let mut app = App::new();
    app.insert_resource(Credits(500))
        .add_message::<BuildRequest>()
        .add_systems(Update, process_build_requests);

    app.world_mut().resource_mut::<Messages<BuildRequest>>()
        .write(BuildRequest { cost: 300 }); // inject a message

    app.update();

    assert_eq!(app.world().resource::<Credits>().0, 200);
}
```

The Bevy traps that bite at runtime

These won't fail to compile — they fail (or silently misbehave) when you run. Know them.

A system that's never registered runs silently never

Forgetting `add_systems` doesn't error; the system simply doesn't run, and your feature is "mysteriously" dead. If logic isn't happening, check registration first.

`single()` on the wrong count

`query.single()` returns `Err` if there are zero or two-plus matches. The 0.18 win is that it's a `Result` you must handle (Chapter 9) rather than a silent panic — so prefer the `?/if let Ok` form and your system degrades gracefully instead of crashing.

Query conflicts: two mutable views of overlapping data

Asking for `Query<&mut Transform, With<Player>>` and `Query<&mut Transform, With<Enemy>>` in one system can panic at startup if Bevy can't prove they're disjoint. The fix is `ParamSet`, which hands out the conflicting queries one at a time.

```
fn sync_positions(mut set: ParamSet<(
    Query<&Transform, With<Player>>, // p0(): read player
    Query<&mut Transform, With<Camera>>, // p1(): write camera
)>) {
    let target = set.p0().single().map(|t| t.translation).unwrap_or_default();
    if let Ok(mut cam) = set.p1().single_mut() {
        cam.translation = target; // follow the player
    }
}
```

MUTATING IN A WAY THAT RE-TRIGGERS `CHANGED`

From Chapter 10: taking `&mut` on a component marks it changed even if you write the same value, which can make a `Changed<T>`-filtered system loop or a test see spurious changes. Guard writes with an equality check when it matters. This is a frequent source of "why does this run every frame?" head-scratching.

Panics in systems take down the app

An `unwrap()` on `None`/`Err` inside a system crashes the whole game. In shipped code, return `Res-ult` from the system and use `?`, or handle the `Option`/`Result` explicitly. Reserve `unwrap()`/`expect()` for tests and setup where a failure *should* abort loudly.

A quick debugging toolbox

- `dbg!(value)` prints a value with its file and line, and returns it — drop it inline anywhere to inspect without restructuring.
- `RUST_LOG` + the **log macros** (Chapter 20) beat `println!` for anything beyond a one-off.
- `RUST_BACKTRACE=1 cargo run` prints a full stack trace on panic so you see *where* it crashed, not just that it did.
- **Bevy inspector crates** (community `bevy-inspector-egui` and friends) give you a live, in-game view of every entity and component — the closest thing to a debugger for ECS state.

What we covered

Built-in unit tests for pure logic, a headless `App`-stepping harness that tests real systems, messages, and state transitions, the runtime traps unique to Bevy (silent unregistered systems, `single()` counts, query conflicts and `ParamSet`, spurious `Changed`), and a practical debugging toolbox. With tests guarding your rules and these traps internalised, you can refactor boldly — which is exactly the confidence Rust promised at the start of this book.

APPENDICES

Appendix A · Python → Rust Cheat Sheet

Keep this page open while you port code. Left is what you'd write in Python; right is the idiomatic Rust.

Syntax and types

Python	Rust
<code>x = 5</code>	<code>let x = 5;</code> (immutable) / <code>let mut x = 5;</code>
<code>x: int = 5</code>	<code>let x: i32 = 5;</code>
<code>MAX = 100</code>	<code>const MAX: i32 = 100;</code>
<code>s = "hi"</code>	<code>let s = "hi";</code> (&str) / <code>String::from("hi")</code> (owned)
<code>f"{a} {b}"</code>	<code>format!("{a} {b}")</code>
<code>None</code>	<code>Option::None</code>
<code>True</code> / <code>False</code>	<code>true</code> / <code>false</code>
<code>x is None</code>	<code>x.is_none()</code>
<code>lambda a, b: a + b</code>	<code> a, b a + b</code>

Control flow and functions

Python	Rust
<code>def f(a: int) -> int:</code>	<code>fn f(a: i32) -> i32 {</code>
<code>return x</code> (early) / last line	<code>return x;</code> / last expr without <code>;</code>
<code>if c: ... elif: ... else:</code>	<code>if c { } else if { } else { }</code>
<code>for i in range(n):</code>	<code>for i in 0..n {</code>
<code>for x in xs:</code>	<code>for x in &xs {</code>
<code>while c:</code>	<code>while c {</code>
<code>match</code> (3.10+)	<code>match</code> (exhaustive, all arms required)

Collections

Python	Rust
<code>[]</code> / <code>list</code>	<code>Vec::new()</code> / <code>vec![]</code>
<code>xs.append(v)</code>	<code>xs.push(v)</code>
<code>xs[i]</code>	<code>xs[i]</code> (panics OOB) / <code>xs.get(i)</code> → <code>Option</code>
<code>len(xs)</code>	<code>xs.len()</code>
<code>{}</code> / <code>dict</code>	<code>HashMap::new()</code>
<code>d[k] = v</code>	<code>map.insert(k, v);</code>
<code>d.get(k, default)</code>	<code>*map.get(&k).unwrap_or(&default)</code>

Python	Rust
<code>d.setdefault(k, 0)</code> then <code>+= 1</code>	<code>*map.entry(k).or_insert(0) += 1;</code>
<code>set()</code>	<code>HashSet::new()</code>
<code>sorted(xs)</code>	<code>xs.sort();</code> (in place)
<code>[f(x) for x in xs if c(x)]</code>	<code>xs.iter().filter(x c(x)).map(f).collect()</code>
<code>sum(xs)</code>	<code>xs.iter().sum()</code>

Errors and absence

Python	Rust
<code>raise ValueError(...)</code>	<code>return Err(...)</code>
<code>try: ... except E:</code>	<code>match result { Ok(v) => ..., Err(e) => ... }</code>
exception propagation	the <code>?</code> operator
<code>x or default</code>	<code>option.unwrap_or(default)</code>
<code>assert cond</code>	<code>assert!(cond)</code> / <code>assert_eq!(a, b)</code>

Classes vs structs

Python	Rust
<code>class C:</code> (data)	<code>struct C { ... }</code>
<code>class C:</code> (methods)	<code>impl C { ... }</code>
<code>self</code>	<code>&self</code> / <code>&mut self</code> / <code>self</code>
inheritance	traits + composition (no inheritance)
<code>@property</code> / duck typing	trait + <code>impl Trait for C</code>
<code>class E(Enum):</code>	<code>enum E { ... }</code> (can hold data per variant)
<code>copy.deepcopy(x)</code>	<code>x.clone()</code> (explicit)

Appendix B · Bevy 0.18 API Quick Reference

A diff-friendly summary of the modern API, with the renames that trip up anyone following older tutorials. When something here disagrees with a blog post, the blog post is probably pre-0.17.

App and scheduling

Task	Code
Create app	<code>App::new().add_plugins(DefaultPlugins)</code>
Run-once system	<code>.add_systems(Startup, sys)</code>
Per-frame system	<code>.add_systems(Update, sys)</code>
Fixed-step system	<code>.add_systems(FixedUpdate, sys)</code>
Order systems	<code>.add_systems(Update, (a, b).chain())</code>
Register a state	<code>.init_state::<S>()</code>
On state enter/exit	<code>.add_systems(OnEnter(S::X), sys) / OnExit</code>
Gate by state	<code>sys.run_if(in_state(S::X))</code>

Entities and components

Task	Code
Define component	<code>#[derive(Component)] struct C;</code>
Spawn	<code>commands.spawn((A, B, C))</code>
Spawn camera (2D)	<code>commands.spawn(Camera2d)</code>
Despawn (recursive)	<code>commands.entity(e).despawn()</code>
Add/remove component	<code>commands.entity(e).insert(C) / .remove::<C>()</code>
Resource	<code>#[derive(Resource)] struct R; + .insert_resource(R)</code>

Queries

Task	Code
Iterate	<code>for c in &query { / &mut query</code>
One entity (Result)	<code>let c = query.single()?; / single_mut()?</code>
Specific entity	<code>query.get(e) / query.get_mut(e) → Result</code>
Filter has/hasn't	<code>Query<&T, (With<A>, Without)></code>
Filter changed/added	<code>Query<&T, Changed<T>> / Added<T></code>
Resolve query conflict	<code>ParamSet<(Query<...>, Query<...>)></code>

Messages vs Events (the big one)

Task	Buffered Message	Observed Event
Define	<code>#[derive(Message)]</code>	<code>#[derive(Event)]</code> / <code>#[derive(EntityEvent)]</code>
Register	<code>app.add_message::<M>()</code>	<code>app.add_observer(sys)</code>
Send	<code>writer.write(m)</code> (<code>MessageWriter</code>)	<code>commands.trigger(e)</code>
Receive	<code>for m in reader.read()</code> (<code>MessageReader</code>)	<code>fn obs(t: On<E>, ...)</code>
Per-entity	—	<code>commands.spawn(..).observe(sys)</code>
Lifecycle	—	<code>On<Add, C></code> / <code>Insert</code> / <code>Replace</code> / <code>Remove</code>

IF YOU REMEMBER ONE THING

Old Bevy and older tutorials say `EventWriter` / `EventReader` / `#[derive(Event)]` / `add_event` for the *buffered* pattern. In 0.18 that is `Message` : `MessageWriter` / `MessageReader` / `#[derive(Message)]` / `add_message`, and you `.write()` instead of `.send()`. The name `Event` now means observer events only.

Renames cheat sheet (old → new)

Old (pre-0.17)	New (0.18)
<code>EventWriter<E></code> / <code>.send(e)</code>	<code>MessageWriter<M></code> / <code>.write(m)</code>
<code>EventReader<E></code>	<code>MessageReader<M></code>
<code>app.add_event::<E>()</code>	<code>app.add_message::<M>()</code>
<code>query.single()</code> (panics)	<code>query.single()</code> → <code>Result</code>
<code>query.get_single()</code>	<code>query.single()</code>
<code>despawn_recursive()</code>	<code>despawn()</code> (now recursive)
<code>time.delta_seconds()</code>	<code>time.delta_secs()</code>
<code>Camera2dBundle</code> / <code>*Bundle</code>	<code>Camera2d</code> + Required Components
<code>rand::thread_rng()</code>	<code>rand::rng()</code>
<code>rng.gen_range(..)</code>	<code>rng.random_range(..)</code>

Appendix C · Cargo.toml Recipes

Start from the closest recipe and trim. All use `edition = "2024"` and the dev-speed profile from Chapter 1. The `2d`, `3d`, and `ui` entries are Bevy 0.18 **feature collections** — opt into only what you ship.

2D action (shmup, point-and-click, card game, visual novel)

```
[dependencies]
bevy = { version = "0.18", default-features = false, features = [
    "2d", "bevy_window", "bevy_winit", "png", "x11",
] }
rand = "0.9"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
ron = "0.8"                # for human-edited config / data tables
```

2D simulation (city builder, RTS)

```
[dependencies]
bevy = { version = "0.18", default-features = false, features = [
    "2d", "bevy_window", "bevy_winit", "png", "x11",
] }
rand = "0.9"
serde = { version = "1", features = ["derive"] }
ron = "0.8"
# Optional: a physics/collision crate when hand-rolled AABBs aren't enough.
# avian2d = "0.3"
```

3D (FPS and beyond)

```
[dependencies]
bevy = { version = "0.18", default-features = false, features = [
    "3d", "bevy_window", "bevy_winit", "tonemapping_luts", "x11",
] }
rand = "0.9"
```

The fast-iteration profile (every project)

```
[profile.dev]
opt-level = 1                # your code: lightly optimised, still debuggable

[profile.dev.package."*"]
opt-level = 3                # dependencies (incl. Bevy): fully optimised once
```

PLATFORM FEATURE

`x11` is for Linux; use `wayland` instead (or both) on Wayland desktops, and omit the platform feature entirely on macOS and Windows. For faster rebuilds during development, add `dynamic_linking` to the feature list — but remove it for release builds and distribution.

Appendix D · The Challenge Ladder

Learning sticks when you build. This is an Advent-of-Code-style ladder: ten bite-size challenges that climb from "hello entity" to a small playable slice, each tied to a concept and a genre from this book. Do them in order. The first three come with full solutions; the rest give you a target and a hint so you can struggle productively — that struggle is where the learning lives.

Day	Challenge	Concept (chapter)	Genre flavour
1	Spawn 100 entities at random positions	spawning, <code>rand</code> 0.9 (2, 19)	any
2	Move them by a <code>Velocity</code> each frame	systems, queries (6)	shmup
3	Despawn any that leave the screen	<code>Commands</code> , despawn (6)	shmup
4	Count entities per zone into a <code>HashMap</code>	<code>entry</code> API (2)	city builder
5	Fire a bullet on Space, pool the bullets	input, pooling (9, 10)	shmup
6	Click to select the nearest unit	cursor→world (9)	RTS
7	A 3-phase turn state machine	states, enums (7)	card game
8	Damage via Message, death via Observer	messages + observers (6, 8)	RTS
9	Save and load the game to JSON	serde, file I/O (19, 20)	any
10	A typewriter dialogue box with choices	strings, UI (15, 17)	visual novel

Day 1 — spawn 100 entities at random positions

```
use bevy::prelude::*;
use rand::Rng;

fn spawn_swarm(mut commands: Commands) {
    commands.spawn(Camera2d);
    let mut rng = rand::rng();
    for _ in 0..100 {
        commands.spawn((
            Transform::from_xyz(
                rng.random_range(-400.0..400.0),
                rng.random_range(-300.0..300.0),
                0.0,
            ),
            Sprite { color: Color::srgb(0.4, 0.8, 1.0),
                custom_size: Some(Vec2::splat(6.0)), ..default() },
        ));
    }
}
```

Day 2 — move them by a Velocity each frame

```
#[derive(Component)]
struct Velocity(Vec2);

// add Velocity(...) to each entity on Day 1, then:
fn drift(time: Res<Time>, mut q: Query<(&mut Transform, &Velocity)>) {
    let dt = time.delta_secs();
```

```

    for (mut tf, v) in &mut q {
        tf.translation += v.0.extend(0.0) * dt;
    }
}

```

Day 3 — despawn entities that leave the screen

```

fn cull(mut commands: Commands, q: Query<Entity, &Transform>) {
    for (e, tf) in &q {
        if tf.translation.x.abs() > 500.0 || tf.translation.y.abs() > 400.0 {
            commands.entity(e).despawn();
        }
    }
}

```

Days 4-10 — targets and hints

- **Day 4.** Build `HashMap<ZoneKind, u32>` by iterating a query and `*map.entry(zone).or_insert(0) += 1;`. *Hint: Chapter 2's grouping idiom.*
- **Day 5.** Spawn a fixed pool of inactive bullets at startup; on Space, find one with `!active` and switch it on. *Hint: Chapter 10's pool; don't spawn per shot.*
- **Day 6.** On left-click, convert the cursor with `viewport_to_world_2d` and mark the nearest unit `Selected`. *Hint: Chapter 13's selection system, trimmed.*
- **Day 7.** An enum `Phase { Draw, Main, End }`, advanced by a `match` on a key press. *Hint: Chapter 14's phase machine.*
- **Day 8.** A `Damage` message reduces `Health`; when HP hits zero, `commands.trigger(Died)` and an observer logs it. *Hint: combine Chapters 6 and 8.*
- **Day 9.** `#[derive(Serialize, Deserialize)]` a save struct; write with `serde_json::to_string_pretty`, read with `from_str`. *Hint: Chapter 20's recipe.*
- **Day 10.** Reveal `full.chars().take(n)` on a timer; render choices as `Text` nodes and pick with number keys. *Hint: Chapters 15 and 17.*

WHEN YOU FINISH DAY 10

You'll have touched spawning, movement, input, collision-adjacent selection, state machines, decoupled events, persistence, and text UI — the connective tissue of every genre in Part VI. Pick the workshop you liked most and turn its chapter into a real, small game.

Appendix E · Where to Go Next

You've covered the language, the engine, the patterns, eight genres, the standard library, practical recipes, and testing. Here's where to deepen each thread.

Official and canonical

- **The Rust Book** — doc.rust-lang.org/book — the definitive language tutorial; revisit the ownership chapters now that you've felt the pain in context.
- **Rust by Example** — doc.rust-lang.org/rust-by-example — runnable snippets for every feature.
- **The Standard Library docs** — doc.rust-lang.org/std — searchable; learn to read the method lists on `Vec`, `Option`, `Result`, `Iterator`.
- **Bevy website & examples** — bevy.org and the `examples/` folder in the Bevy repo — the most reliable, version-correct reference. Every example compiles against its Bevy version.
- **Bevy migration guides** — bevy.org/learn/migration-guides — keep this open when you upgrade; it lists every rename like the ones in Appendix B.

Community references

- **The Bevy Cheatbook** — a community guide organised by task ("how do I..."); excellent for the practical glue between engine features.
- **Rust Design Patterns** — rust-unofficial.github.io/patterns — the catalogue Chapter 11 drew from; read the idioms section.
- **This Week in Bevy** — a weekly digest of engine changes and community projects; the easiest way to track a fast-moving pre-1.0 engine.

Crates worth knowing

- `thiserror` / `anyhow` — ergonomic error handling (Chapter 19).
- `serde` + `ron` / `serde_json` — serialisation for saves, config, data tables.
- `rand` 0.9 — randomness, with the seeding pattern from Chapter 20.
- **A physics crate** (e.g. an Avian-style 2D/3D physics library) when hand-rolled collision stops scaling.
- **An inspector** (community `bevy-inspector-egui`-style tools) for live ECS debugging.

A closing word

Rust's compiler felt like an adversary on page one. By now it should feel like what it is: a tireless reviewer that catches the bugs you'd otherwise ship. Every `error[E0382]` you fixed was a crash that never happened. Build something small, finish it, and let the borrow checker keep having your back.

Now go clone a classic. — *Happy hacking.*